

Python Interview Questions

Scripting & Automation

Total Questions:	100
Topics Covered:	File I/O, Subprocess, Regex, Web Scraping, APIs, Scheduling, Concurrency, CLI Tools, Logging, Error Handling, OS Automation & More
Level:	Beginner to Advanced
Format:	Question + Explanation + Full Code Example

Complete with working code examples for each question

Table of Contents

1. File & Directory Operations	Q1–Q10
2. Process & Subprocess Automation	Q11–Q20
3. Regular Expressions	Q21–Q28
4. Web Scraping & HTTP Automation	Q29–Q38
5. Scheduling & Task Automation	Q39–Q45
6. Concurrency & Parallelism	Q46–Q54
7. Logging & Error Handling	Q55–Q63
8. Config, Environment & CLI	Q64–Q72
9. OS & System Automation	Q73–Q81
10. Database & File Format Automation	Q82–Q90
11. Advanced Scripting Patterns	Q91–Q100

Section 1: File & Directory Operations (Q1–Q10)

Q1. How do you read a file line by line efficiently in Python?

Use a for loop with the file object. This is memory-efficient as it reads one line at a time without loading the entire file.

```
# Method 1: Using for loop (most efficient)
with open('data.txt', 'r') as f:
    for line in f:
        print(line.strip())

# Method 2: readlines() - loads all into memory
with open('data.txt', 'r') as f:
    lines = f.readlines()
    for line in lines:
        print(line.strip())

# Method 3: readline() - one at a time
with open('data.txt', 'r') as f:
    while True:
        line = f.readline()
        if not line:
            break
        print(line.strip())
```

Q2. How do you write to a file and append to an existing file?

Use mode 'w' to write (overwrites) and mode 'a' to append to an existing file.

```
# Write to file (creates or overwrites)
with open('output.txt', 'w') as f:
    f.write('Hello, World!\n')
    f.writelines(['Line 1\n', 'Line 2\n', 'Line 3\n'])

# Append to file
with open('output.txt', 'a') as f:
    f.write('This line is appended.\n')

# Write binary data
with open('data.bin', 'wb') as f:
    f.write(b'\x00\x01\x02\x03')

# Read binary data
with open('data.bin', 'rb') as f:
    data = f.read()
    print(list(data)) # [0, 1, 2, 3]
```

Q3. How do you traverse a directory tree and list all files?

Use `os.walk()` for recursive traversal or `pathlib.Path.rglob()` for pattern-based recursive searches.

```

import os
from pathlib import Path

# Method 1: os.walk() - traverse directory tree
for root, dirs, files in os.walk('/home/user/project'):
    for file in files:
        full_path = os.path.join(root, file)
        print(full_path)

# Method 2: pathlib rglob - find all .py files recursively
py_files = list(Path('/home/user/project').rglob('*.py'))
for f in py_files:
    print(f)

# Method 3: List only immediate directory contents
p = Path('/home/user/project')
for item in p.iterdir():
    if item.is_file():
        print(f'File: {item.name}')
    elif item.is_dir():
        print(f'Dir: {item.name}')

```

Q4. How do you copy, move, and delete files using Python?

Use the shutil module for copy/move operations and os or pathlib for deletion.

```

import shutil
import os
from pathlib import Path

# Copy file
shutil.copy('source.txt', 'destination.txt') # copies file
shutil.copy2('source.txt', 'destination.txt') # copies with metadata

# Copy entire directory tree
shutil.copytree('src_folder', 'dst_folder')

# Move file or directory
shutil.move('old_path.txt', 'new_path.txt')

# Delete a file
os.remove('file_to_delete.txt')
Path('file_to_delete.txt').unlink() # pathlib way

# Delete empty directory
os.rmdir('empty_folder')

# Delete directory and all contents (CAREFUL!)
shutil.rmtree('folder_to_delete')

# Safe delete (check existence first)

```

```

file = Path('maybe_exists.txt')
if file.exists():
    file.unlink()
    print('Deleted')
else:
    print('File not found')

```

Q5. How do you get file metadata like size, creation time, and modification time?

Use `os.stat()` or `pathlib`'s `stat()` method to retrieve file metadata.

```

import os
import time
from pathlib import Path
from datetime import datetime

file_path = 'example.txt'

# Using os.stat()
stat = os.stat(file_path)
print(f'Size: {stat.st_size} bytes')
print(f'Last modified: {datetime.fromtimestamp(stat.st_mtime)}')
print(f'Last accessed: {datetime.fromtimestamp(stat.st_atime)}')
print(f'Created: {datetime.fromtimestamp(stat.st_ctime)}')

# Using pathlib
p = Path(file_path)
stat = p.stat()
print(f'Size: {stat.st_size} bytes')
print(f'Extension: {p.suffix}')
print(f'Stem: {p.stem}')
print(f'Parent: {p.parent}')

# Find files modified in last 24 hours
cutoff = time.time() - 86400
recent = [f for f in Path('.').rglob('*')
if f.is_file() and f.stat().st_mtime > cutoff]
print(f'Files modified in last 24h: {len(recent)}')

```

Q6. How do you create temporary files and directories?

Use the `tempfile` module to create temporary files and directories that are auto-cleaned.

```

import tempfile
import os

# Create a temporary file
with tempfile.NamedTemporaryFile(mode='w', suffix='.txt', delete=False) as tmp:
    tmp.write('Temporary content')
    tmp_path = tmp.name
print(f'Temp file: {tmp_path}')

```

```

os.unlink(tmp_path) # Manual cleanup when delete=False

# Auto-deleted temporary file
with tempfile.TemporaryFile(mode='w+') as tmp:
    tmp.write('Auto-deleted content')
    tmp.seek(0)
    print(tmp.read())
# File is deleted here automatically

# Temporary directory
with tempfile.TemporaryDirectory() as tmp_dir:
    print(f'Temp dir: {tmp_dir}')
    temp_file = os.path.join(tmp_dir, 'test.txt')
    with open(temp_file, 'w') as f:
        f.write('test')
# Directory and all contents deleted automatically

```

Q7. How do you work with JSON files in Python?

Use the json module to serialize (dump) and deserialize (load) JSON data.

```

import json
from pathlib import Path

data = {
    'name': 'Alice',
    'age': 30,
    'languages': ['Python', 'JavaScript'],
    'active': True
}

# Write JSON to file
with open('data.json', 'w') as f:
    json.dump(data, f, indent=4)

# Read JSON from file
with open('data.json', 'r') as f:
    loaded = json.load(f)
    print(loaded['name']) # Alice

# Convert dict to JSON string
json_str = json.dumps(data, indent=2)
print(json_str)

# Parse JSON string
parsed = json.loads(json_str)
print(type(parsed)) # dict

# Handle files with pathlib

```

```

path = Path('config.json')
if path.exists():
    config = json.loads(path.read_text())
else:
    config = {}
path.write_text(json.dumps(config, indent=2))

```

Q8. How do you work with CSV files in Python?

Use the csv module for reading/writing CSV files. For large datasets, pandas is preferred.

```

import csv

# Write CSV
rows = [
    ['Name', 'Age', 'City'],
    ['Alice', 30, 'NYC'],
    ['Bob', 25, 'LA'],
]
with open('people.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(rows)

# Read CSV as list of lists
with open('people.csv', 'r') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)

# Read CSV as list of dicts
with open('people.csv', 'r') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row['Name'], row['Age'])

# Write CSV from dicts
records = [{'Name': 'Carol', 'Age': 28, 'City': 'Chicago'}]
with open('output.csv', 'w', newline='') as f:
    fields = ['Name', 'Age', 'City']
    writer = csv.DictWriter(f, fieldnames=fields)
    writer.writeheader()
    writer.writerows(records)

```

Q9. How do you use pathlib for file path operations?

pathlib provides an object-oriented interface for filesystem paths, much cleaner than os.path.

```

from pathlib import Path

# Create path
p = Path('/home/user/documents/report.pdf')

```

```

# Path components
print(p.name) # report.pdf
print(p.stem) # report
print(p.suffix) # .pdf
print(p.parent) # /home/user/documents
print(p.parts) # ('/', 'home', 'user', 'documents', 'report.pdf')

# Build paths
base = Path.home()
config = base / '.config' / 'myapp' / 'settings.json'
print(config)

# Create directory (including parents)
new_dir = Path('output/data/2024')
new_dir.mkdir(parents=True, exist_ok=True)

# Check existence
print(p.exists())
print(p.is_file())
print(p.is_dir())

# Read/write text
Path('hello.txt').write_text('Hello World')
content = Path('hello.txt').read_text()

# Glob patterns
py_files = list(Path('.').glob('**/*.py'))
log_files = list(Path('/var/log').glob('*.log'))

# Rename / resolve
p.rename(p.with_suffix('.txt')) # change extension

```

Q10. How do you zip and unzip files with Python?

Use the zipfile module for zip operations and shutil.make_archive for high-level archiving.

```

import zipfile
import shutil
from pathlib import Path

# Create a zip file
with zipfile.ZipFile('archive.zip', 'w', zipfile.ZIP_DEFLATED) as zf:
    zf.write('file1.txt')
    zf.write('file2.txt', 'renamed_in_zip.txt')
# Add entire folder
for f in Path('my_folder').rglob('*'):
    if f.is_file():
        zf.write(f, f.relative_to('.'))

```



```
# Extract all files
with zipfile.ZipFile('archive.zip', 'r') as zf:
    zf.extractall('output_folder')

# Extract specific file
with zipfile.ZipFile('archive.zip', 'r') as zf:
    zf.extract('file1.txt', 'output_folder')

# List contents
with zipfile.ZipFile('archive.zip', 'r') as zf:
    print(zf.namelist())

# High-level archiving
shutil.make_archive('backup', 'zip', 'my_folder')
shutil.make_archive('backup', 'tar', 'my_folder')
shutil.unpack_archive('archive.zip', 'extracted/')
```

Section 2: Process & Subprocess Automation (Q11–Q20)

Q11. What is the difference between `os.system()`, `subprocess.run()`, and `subprocess.Popen()`?

`os.system()` is simple but limited. `subprocess.run()` is modern and preferred. `Popen()` gives full control for complex scenarios.

```
import os
import subprocess

# os.system() - simple, returns exit code only
ret = os.system('ls -la')
print(f'Exit code: {ret}')
```



```
# subprocess.run() - modern, recommended
result = subprocess.run(['ls', '-la'], capture_output=True, text=True)
print(result.stdout)
print(result.returncode)
```



```
# subprocess.run() with shell=True (use with caution)
result = subprocess.run('echo Hello World', shell=True,
capture_output=True, text=True)
print(result.stdout)
```



```
# subprocess.Popen() - full control, non-blocking
proc = subprocess.Popen(['ping', '-c', '3', 'google.com'],
stdout=subprocess.PIPE,
stderr=subprocess.PIPE,
text=True)
stdout, stderr = proc.communicate(timeout=10)
print(stdout)
print(f'Return code: {proc.returncode}')
```

Q12. How do you capture and process the output of a subprocess in real time?

Use `Popen` with `iter()` to read `stdout` line by line as the process runs.

```
import subprocess

# Real-time output reading
def run_with_live_output(cmd):
    proc = subprocess.Popen(
        cmd,
        stdout=subprocess.PIPE,
        stderr=subprocess.STDOUT,
        text=True,
        bufsize=1
    )
    lines = []
    for line in iter(proc.stdout.readline, ''):
        line = line.rstrip()
```

```

print(line)
lines.append(line)
proc.wait()
return proc.returncode, lines

# Usage
ret, output = run_with_live_output(['python3', '-c',
'import time; [print(i) or time.sleep(0.1) for i in range(5)]'])
print(f'Exit: {ret}')

```

Q13. How do you run a subprocess with a timeout?

Pass the timeout parameter to `subprocess.run()` or `Popen.communicate()` to avoid hanging indefinitely.

```

import subprocess

# Timeout with subprocess.run()
try:
    result = subprocess.run(
        ['sleep', '10'],
        timeout=3,
        capture_output=True,
        text=True
    )
except subprocess.TimeoutExpired as e:
    print(f'Process timed out after {e.timeout}s')

# Timeout with Popen
proc = subprocess.Popen(['sleep', '10'])
try:
    proc.communicate(timeout=3)
except subprocess.TimeoutExpired:
    proc.kill()
    proc.communicate() # Clean up
    print('Process killed after timeout')

# Run with retry on timeout
def run_with_retry(cmd, timeout=30, retries=3):
    for attempt in range(retries):
        try:
            result = subprocess.run(cmd, timeout=timeout,
                capture_output=True, text=True,
                check=True)
            return result.stdout
        except subprocess.TimeoutExpired:
            print(f'Attempt {attempt+1} timed out, retrying...')
    except subprocess.CalledProcessError as e:
        print(f'Command failed: {e.stderr}')
        break
    return None

```

Q14. How do you chain multiple subprocess commands using pipes?

Use multiple Popen instances and pipe stdout of one to stdin of the next, mimicking shell pipes.

```
import subprocess

# Equivalent to: cat file.txt | grep error | wc -l
p1 = subprocess.Popen(['cat', 'file.txt'], stdout=subprocess.PIPE)
p2 = subprocess.Popen(['grep', 'error'], stdin=p1.stdout,
                      stdout=subprocess.PIPE)
p3 = subprocess.Popen(['wc', '-l'], stdin=p2.stdout,
                      stdout=subprocess.PIPE, text=True)

p1.stdout.close() # Allow p1 to receive SIGPIPE if p2 exits
p2.stdout.close()
output, _ = p3.communicate()
print(f'Error line count: {output.strip()}')

# Simpler: use shell=True for simple pipe chains
result = subprocess.run(
    "cat file.txt | grep error | wc -l",
    shell=True, capture_output=True, text=True
)
print(result.stdout.strip())
```

Q15. How do you set environment variables for a subprocess?

Pass an env dict to `subprocess.run()` or `Popen`. Use `os.environ.copy()` as the base to preserve existing vars.

```
import subprocess

import os

# Pass custom environment
env = os.environ.copy()
env['MY_VAR'] = 'hello'
env['DEBUG'] = '1'

result = subprocess.run(
    ['python3', '-c', 'import os; print(os.environ.get("MY_VAR"))'],
    env=env,
    capture_output=True,
    text=True
)
print(result.stdout) # hello

# Run with minimal environment
minimal_env = {
    'PATH': '/usr/bin:/bin',
    'HOME': '/tmp'
}
result = subprocess.run(['env'], env=minimal_env,
    capture_output=True, text=True)
```

```

print(result.stdout)

# Inherit + override specific vars
env = {'**os.environ', 'DATABASE_URL': 'postgres://localhost/mydb'}
result = subprocess.run(['my_app'], env=env)

```

Q16. How do you run a subprocess in a different working directory?

Use the `cwd` parameter in `subprocess.run()` or `Popen` to set the working directory.

```

import subprocess
import tempfile
import os

# Run in specific directory
result = subprocess.run(
    ['ls', '-la'],
    cwd='/tmp',
    capture_output=True,
    text=True
)
print(result.stdout)

# Run git command in a repo directory
def git_status(repo_path):
    result = subprocess.run(
        ['git', 'status', '--short'],
        cwd=repo_path,
        capture_output=True,
        text=True
    )
    return result.stdout.strip()

# Run in temp directory
with tempfile.TemporaryDirectory() as tmp:
    # Initialize git repo
    subprocess.run(['git', 'init'], cwd=tmp, check=True,
        capture_output=True)
    print(f'Git initialized in {tmp}')
    print(git_status(tmp))

```

Q17. How do you check if a command/program exists before running it?

Use `shutil.which()` which returns the full path if the command exists, or `None` if not found.

```

import shutil
import subprocess
import sys

# Check if command exists
def command_exists(cmd):

```

```

return shutil.which(cmd) is not None

# Usage
if command_exists('git'):
    result = subprocess.run(['git', '--version'],
        capture_output=True, text=True)
    print(result.stdout.strip())
else:
    print('git is not installed')

# Find location of command
git_path = shutil.which('python3')
print(f'Python is at: {git_path}')

# Require dependencies
required = ['git', 'curl', 'docker']
missing = [cmd for cmd in required if not command_exists(cmd)]
if missing:
    print(f'Missing tools: {", ".join(missing)}')
    sys.exit(1)
else:
    print('All dependencies found')

```

Q18. How do you run multiple subprocesses concurrently?

Start multiple Popen processes and collect results using communicate() or use ThreadPoolExecutor.

```

import subprocess
from concurrent.futures import ThreadPoolExecutor, as_completed

commands = [
    ['ping', '-c', '2', 'google.com'],
    ['ping', '-c', '2', 'github.com'],
    ['ping', '-c', '2', 'python.org'],
]

# Method 1: Popen list
procs = [subprocess.Popen(cmd, stdout=subprocess.PIPE,
    stderr=subprocess.PIPE, text=True)
    for cmd in commands]
results = [proc.communicate() for proc in procs]
for (stdout, stderr), cmd in zip(results, commands):
    print(f'{cmd[-1]}: {"OK" if stdout else "FAIL"}')

# Method 2: ThreadPoolExecutor
def run_cmd(cmd):
    result = subprocess.run(cmd, capture_output=True,
        text=True, timeout=10)
    return result.returncode, result.stdout

```

```

with ThreadPoolExecutor(max_workers=4) as executor:
    futures = {executor.submit(run_cmd, cmd): cmd for cmd in commands}
    for future in as_completed(futures):
        cmd = futures[future]
        code, out = future.result()
        print(f'{cmd[-1]}: returncode={code}')

```

Q19. How do you handle subprocess errors gracefully?

Use `check=True` to raise `CalledProcessError` on non-zero exit, and handle exceptions appropriately.

```

import subprocess
import logging

logging.basicConfig(level=logging.INFO)

def safe_run(cmd, **kwargs):
    """Run a command safely with comprehensive error handling."""
    try:
        result = subprocess.run(
            cmd,
            capture_output=True,
            text=True,
            check=True, # raises CalledProcessError on failure
            **kwargs
        )
        return result.stdout.strip()
    except subprocess.CalledProcessError as e:
        logging.error(f'Command failed: {" ".join(cmd)}')
        logging.error(f'Exit code: {e.returncode}')
        logging.error(f'Stderr: {e.stderr}')
        return None
    except FileNotFoundError:
        logging.error(f'Command not found: {cmd[0]}')
        return None
    except subprocess.TimeoutExpired:
        logging.error(f'Command timed out: {" ".join(cmd)}')
        return None

output = safe_run(['ls', '-la', '/tmp'])
if output:
    print(output)

```

Q20. How do you send input to an interactive subprocess?

Use `subprocess.communicate(input=...)` to send stdin data, or use `Popen.stdin.write()` for interactive processes.

```

import subprocess

# Send input via communicate()
result = subprocess.run(

```

```

['python3', '-c', 'name = input("Name: "); print(f"Hello {name}")'],
input='Alice',
capture_output=True,
text=True
)
print(result.stdout) # Hello Alice

# Feed multiple inputs
proc = subprocess.run(
['cat'], # cat echoes stdin to stdout
input='Hello\nWorld\n',
capture_output=True,
text=True
)
print(proc.stdout)

# Interactive process with Popen
proc = subprocess.Popen(
['python3', '-c', '''
import sys
for line in sys.stdin:
print(f"Got: {line.rstrip()}")
'''],
stdin=subprocess.PIPE,
stdout=subprocess.PIPE,
text=True
)
stdout, _ = proc.communicate(input='line1\nline2\nline3\n')
print(stdout)

```


Section 3: Regular Expressions (Q21–Q28)

Q21. What is the difference between `re.match()`, `re.search()`, and `re.fullmatch()`?

`match()` checks only the beginning, `search()` finds anywhere in string, `fullmatch()` requires entire string to match.

```
import re

text = 'Hello World 123'

# re.match() - matches at the BEGINNING only
m = re.match(r'Hello', text)
print(m.group()) # Hello

m = re.match(r'World', text)
print(m) # None - not at beginning

# re.search() - matches ANYWHERE in the string
m = re.search(r'World', text)
print(m.group()) # World

# re.fullmatch() - ENTIRE string must match
m = re.fullmatch(r'Hello World 123', text)
print(m.group()) # Hello World 123

m = re.fullmatch(r'Hello', text)
print(m) # None - doesn't match full string

# re.findall() - returns list of all matches
matches = re.findall(r'\d+', 'abc 123 def 456')
print(matches) # ['123', '456']

# re.finditer() - returns iterator of match objects
for m in re.finditer(r'\d+', 'abc 123 def 456'):
    print(m.group(), m.start(), m.end())
```

Q22. How do you use groups in regular expressions?

Use parentheses `()` for capturing groups. Named groups (`?P...`) improve readability.

```
import re

# Numbered groups
m = re.match(r'(\d{4})-(\d{2})-(\d{2})', '2024-03-15')
if m:
    print(m.group(0)) # 2024-03-15 (entire match)
    print(m.group(1)) # 2024 (year)
    print(m.group(2)) # 03 (month)
    print(m.group(3)) # 15 (day)
    print(m.groups()) # ('2024', '03', '15')
```

```

# Named groups
pattern = r'(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})'
m = re.match(pattern, '2024-03-15')
if m:
    print(m.group('year')) # 2024
    print(m.groupdict()) # {'year': '2024', ...}

# Non-capturing group (?:...)
m = re.match(r'(?:https?://)?(\w+\.\w+)', 'https://example.com')
print(m.group(1)) # example.com (scheme not captured)

# Backreferences
m = re.search(r'(\w+) \1', 'hello hello world')
print(m.group()) # hello hello

```

Q23. How do you perform search and replace with regex?

Use `re.sub()` for replacement. The replacement can be a string or a function for dynamic replacements.

```

import re

text = 'Phone: 123-456-7890 or (098) 765-4321'

# Simple substitution
result = re.sub(r'\d', 'X', text)
print(result) # Phone: XXX-XXX-XXXX or (XXX) XXX-XXXX

# Limit number of replacements
result = re.sub(r'\d', 'X', text, count=5)
print(result)

# Use backreference in replacement
dates = 'Date: 2024-03-15'
# Reformat from YYYY-MM-DD to DD/MM/YYYY
result = re.sub(r'(\d{4})-(\d{2})-(\d{2})', r'\3/\2/\1', dates)
print(result) # Date: 15/03/2024

# Function as replacement (capitalize each word)
def capitalize_match(m):
    return m.group().upper()

result = re.sub(r'\b\w{4}\b', capitalize_match, 'hello world from python')
print(result)

# re.subn() returns (new_string, count)
result, n = re.subn(r'\d+', 'NUM', 'a1 b22 c333')
print(f'{result} (replaced {n} times)')

```

Q24. How do you compile a regex and use flags?

Use `re.compile()` to pre-compile patterns for reuse. Flags like `re.IGNORECASE`, `re.MULTILINE` modify behavior.

```
import re

# Compile for reuse (faster in loops)
email_pattern = re.compile(
    r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
)

emails = ['user@example.com', 'bad.email', 'test@test.org']
for e in emails:
    print(f'{e}: {bool(email_pattern.match(e))}')

# Case-insensitive flag
m = re.search(r'hello', 'Hello World', re.IGNORECASE)
print(m.group()) # Hello

# MULTILINE: ^ and $ match at start/end of each line
text = 'line1\nline2\nline3'
matches = re.findall(r'^line\d', text, re.MULTILINE)
print(matches) # ['line1', 'line2', 'line3']

# DOTALL: . matches newlines too
m = re.search(r'start.+end', 'start\nmiddle\nend', re.DOTALL)
print(m.group())

# VERBOSE: ignore whitespace, allow comments
pattern = re.compile(r'''
    \d{3} # area code
    [-. ]? # separator
    \d{3} # prefix
    [-. ]? # separator
    \d{4} # line number
''', re.VERBOSE)
print(bool(pattern.match('123-456-7890')))
```

Q25. How do you validate common patterns like emails, URLs, and IP addresses?

Write specific regex patterns for each format. For production use, consider validation libraries.

```
import re

def validate_email(email):
    pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
    return bool(re.match(pattern, email))

def validate_ipv4(ip):
    pattern = r'^(25[0-5]|2[0-4]\d|[01]?[0-9]\d?)\.'
    pattern += r'(25[0-5]|2[0-4]\d|[01]?[0-9]\d?)\.'
    pattern += r'(25[0-5]|2[0-4]\d|[01]?[0-9]\d?)\.'
    pattern += r'(25[0-5]|2[0-4]\d|[01]?[0-9]\d?)$'
```

```

return bool(re.match(pattern, ip))

def validate_url(url):
    pattern = r'https?://[\w\.-]+(\.[\w\.-]+)+([\w.,@?^=%&:/~+#\.-]*)?'
    return bool(re.match(pattern, url))

def validate_phone(phone):
    # Matches: 123-456-7890, (123) 456-7890, 1234567890
    pattern = r'^(\+?1[-. ]?)?(\(?d{3}\)?[-. ]?)d{3}[-. ]?d{4}$'
    return bool(re.match(pattern, phone))

tests = [
    ('alice@example.com', validate_email),
    ('192.168.1.1', validate_ipv4),
    ('https://www.python.org', validate_url),
    ('123-456-7890', validate_phone),
]
for value, fn in tests:
    print(f'{fn.__name__}({value!r}): {fn(value)!r}')

```

Q26. What is greedy vs non-greedy (lazy) matching?

Greedy quantifiers (*, +, ?) match as much as possible. Add '?' to make them lazy (match as little as possible).

```

import re

html = '<b>Hello</b> and <b>World</b>'

# Greedy - matches as MUCH as possible
m = re.search(r'<b>.*</b>', html)
print(m.group()) # <b>Hello</b> and <b>World</b> (too much!)

# Non-greedy/Lazy - matches as LITTLE as possible
m = re.search(r'<b>.*?</b>', html)
print(m.group()) # <b>Hello</b> (correct!)

# Extract all bold tags
greedy = re.findall(r'<b>.*</b>', html)
lazy = re.findall(r'<b>.*?</b>', html)
print(f'Greedy: {greedy}') # entire string
print(f'Lazy: {lazy}') # ['<b>Hello</b>', '<b>World</b>']

# Greedy vs lazy with +
text = 'aXXXb aXb'
print(re.findall(r'a.+b', text)) # ['aXXXb aXb'] greedy
print(re.findall(r'a.+?b', text)) # ['aXXXb', 'aXb'] lazy

# Possessive equivalent - use atomic group workaround
# Python 3.11+: supports atomic groups (?>...)
pattern = r'".*?"' # Extract quoted strings

```

```
quoted = re.findall(pattern, 'Say "hello" and "world"')
print(quoted) # ['"hello"', '"world"']
```

Q27. How do you extract data from structured text using regex?

Combine named groups and finditer to extract structured data from logs or text files.

```
import re
from datetime import datetime

# Extract data from log file
log_pattern = re.compile(
    r'(?P<timestamp>\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}) '
    r'(?P<level>\w+) '
    r'(?P<message>.+)'
)

logs = [
    '2024-03-15 10:30:00 ERROR Database connection failed',
    '2024-03-15 10:31:05 INFO Server started on port 8080',
    '2024-03-15 10:32:00 WARNING High memory usage: 85%',
]

for log in logs:
    m = log_pattern.match(log)
    if m:
        print(f"Level: {m.group('level')}")
        print(f"Message: {m.group('message')}")
        print()

# Extract key=value pairs
config_text = 'HOST=localhost PORT=5432 DEBUG=True'
pairs = re.findall(r'(\w+)=([\S+)]', config_text)
config = dict(pairs)
print(config) # {'HOST': 'localhost', 'PORT': '5432', ...}

# Extract all numbers from text
text = 'Sales: 1,234.56 units at $29.99 each = $36,952.44'
numbers = re.findall(r'[\d,]+\.\d*', text)
print(numbers) # ['1,234.56', '29.99', '36,952.44']
```

Q28. How do you use lookahead and lookbehind assertions?

Lookahead (?=...) and lookbehind (?<=...) match positions without consuming characters.

```
import re

# Positive lookahead (?=...) - match if followed by pattern
text = 'python3 java8 ruby python2'
# Match word only if followed by digit
matches = re.findall(r'\w+(?=\d)', text)
```

```
print(matches) # ['python', 'java', 'ruby', 'python']

# Negative lookahead (?!\...) - match if NOT followed by pattern
matches = re.findall(r'\w+(?!\d)\b', 'cat cats cat9')
print(matches)

# Positive lookbehind (?<=...) - match if preceded by pattern
prices = '$10 EUR20 $30 EUR40'
# Get numbers preceded by $
usd = re.findall(r'(?<=\$)\d+', prices)
print(usd) # ['10', '30']

# Get numbers preceded by EUR
eur = re.findall(r'(?<=EUR)\d+', prices)
print(eur) # ['20', '40']

# Negative lookbehind (?<!\...)
# Words not preceded by @
text = 'hello @mention world @another test'
words = re.findall(r'(?<!\@)\b\w+\b', text)
print(words) # ['hello', 'world', 'test']
```

Section 4: Web Scraping & HTTP Automation (Q29–Q38)

Q29. How do you make HTTP GET and POST requests with the requests library?

The requests library provides a simple interface for HTTP requests with automatic JSON handling.

```
import requests

# GET request
response = requests.get('https://api.github.com/users/octocat')
print(response.status_code) # 200
print(response.headers['content-type'])
data = response.json() # Parse JSON
print(data['login'])

# GET with query parameters
params = {'q': 'python', 'sort': 'stars', 'order': 'desc'}
response = requests.get('https://api.github.com/search/repositories',
params=params)
results = response.json()
print(results['total_count'])

# POST with JSON body
payload = {'username': 'alice', 'email': 'alice@example.com'}
response = requests.post(
'https://httpbin.org/post',
json=payload, # auto-sets Content-Type header
headers={'Authorization': 'Bearer my-token'}
)
print(response.json())

# POST with form data
response = requests.post(
'https://httpbin.org/post',
data={'key': 'value', 'other': 'data'}
)
```

Q30. How do you handle HTTP errors and retries?

Use `response.raise_for_status()` and implement retry logic with exponential backoff.

```
import requests
import time
import logging
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

def create_session_with_retries(retries=3, backoff=0.3):
    session = requests.Session()
    retry = Retry(
total=retries,
```

```

backoff_factor=backoff, # 0.3, 0.6, 1.2 seconds
status_forcelist=[500, 502, 503, 504],
)
adapter = HTTPAdapter(max_retries=retry)
session.mount('http://', adapter)
session.mount('https://', adapter)
return session

session = create_session_with_retries()

try:
response = session.get('https://api.example.com/data', timeout=10)
response.raise_for_status() # Raises for 4xx/5xx
data = response.json()
except requests.exceptions.HTTPError as e:
logging.error(f'HTTP error: {e.response.status_code} - {e}')
except requests.exceptions.ConnectionError as e:
logging.error(f'Connection error: {e}')
except requests.exceptions.Timeout:
logging.error('Request timed out')
except requests.exceptions.RequestException as e:
logging.error(f'Unexpected error: {e}')

```

Q31. How do you use sessions and cookies with requests?

Use requests.Session() to persist cookies, headers, and connection pools across multiple requests.

```

import requests

# Session persists cookies and headers
session = requests.Session()
session.headers.update({
    'User-Agent': 'MyBot/1.0',
    'Accept': 'application/json',
})

# Login (cookies stored automatically)
login_data = {'username': 'alice', 'password': 'secret'}
session.post('https://example.com/login', data=login_data)

# Subsequent requests carry the session cookie
response = session.get('https://example.com/profile')
print(response.json())

# Inspect cookies
print(session.cookies.get_dict())

# Manual cookie handling
session.cookies.set('custom_cookie', 'value')

```



```

# Use session as context manager
with requests.Session() as s:
    s.auth = ('user', 'password') # Basic auth
    r = s.get('https://api.example.com/protected')
    print(r.json())

# Download a file with streaming
with session.get('https://example.com/bigfile.zip',
stream=True) as r:
    r.raise_for_status()
    with open('bigfile.zip', 'wb') as f:
        for chunk in r.iter_content(chunk_size=8192):
            f.write(chunk)

```

Q32. How do you parse HTML with BeautifulSoup?

BeautifulSoup parses HTML/XML documents and provides methods to search and navigate the DOM tree.

```

from bs4 import BeautifulSoup
import requests

html = '''
<html><body>
<h1 class="title">Python Tutorial</h1>
<ul id="topics">
<li><a href="/basics">Basics</a></li>
<li><a href="/advanced">Advanced</a></li>
</ul>
<div class="content">
<p>Learn Python scripting.</p>
</div>
</body></html>
'''

soup = BeautifulSoup(html, 'html.parser')

# Find elements
title = soup.find('h1')
print(title.text) # Python Tutorial
print(title['class']) # ['title']

# Find all links
links = soup.find_all('a')
for link in links:
    print(link.text, '->', link.get('href'))

# CSS selectors
items = soup.select('#topics li a')
for item in items:
    print(item.text)

```

```
# Navigate tree
ul = soup.find('ul', id='topics')
first_li = ul.find('li')
print(first_li.text)
```

Q33. How do you scrape a website while respecting robots.txt and rate limits?

Check robots.txt, add delays between requests, and set a descriptive User-Agent. Use urllib.robotparser.

```
import requests
import time
import random
from urllib.robotparser import RobotFileParser
from urllib.parse import urljoin

class PoliteScraper:
    def __init__(self, base_url, delay=(1, 3)):
        self.base_url = base_url
        self.delay = delay
        self.session = requests.Session()
        self.session.headers['User-Agent'] = 'MyBot/1.0 (+mailto:me@example.com)'
        self._load_robots()

    def _load_robots(self):
        self.rp = RobotFileParser()
        robots_url = urljoin(self.base_url, '/robots.txt')
        self.rp.set_url(robots_url)
        try:
            self.rp.read()
        except Exception:
            pass # If no robots.txt, allow all

    def can_fetch(self, url):
        return self.rp.can_fetch('*', url)

    def get(self, url):
        if not self.can_fetch(url):
            print(f'Blocked by robots.txt: {url}')
            return None
        time.sleep(random.uniform(*self.delay)) # Polite delay
        try:
            return self.session.get(url, timeout=10)
        except requests.RequestException as e:
            print(f'Error fetching {url}: {e}')
            return None

scraper = PoliteScraper('https://example.com')
resp = scraper.get('https://example.com/page')
```

Q34. How do you handle JavaScript-rendered pages with Selenium?

Use Selenium WebDriver to control a browser that executes JavaScript before scraping the rendered HTML.

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.chrome.options import Options

# Configure headless Chrome
options = Options()
options.add_argument('--headless')
options.add_argument('--no-sandbox')
options.add_argument('--disable-dev-shm-usage')

driver = webdriver.Chrome(options=options)

try:
    driver.get('https://example.com')

    # Wait for element to appear (up to 10 seconds)
    wait = WebDriverWait(driver, 10)
    element = wait.until(
        EC.presence_of_element_located((By.ID, 'main-content'))
    )

    # Click a button
    btn = driver.find_element(By.CSS_SELECTOR, 'button.load-more')
    btn.click()

    # Fill form
    field = driver.find_element(By.NAME, 'search')
    field.send_keys('python tutorial')

    # Extract text
    items = driver.find_elements(By.CLASS_NAME, 'result')
    for item in items:
        print(item.text)

    # Execute JavaScript
    driver.execute_script('window.scrollTo(0, document.body.scrollHeight);')

finally:
    driver.quit()
```

Q35. How do you download multiple files concurrently?

Use ThreadPoolExecutor with requests to download multiple files in parallel efficiently.

```
import requests
import os
```

```

from concurrent.futures import ThreadPoolExecutor, as_completed
from pathlib import Path

def download_file(url, dest_folder='downloads'):
    Path(dest_folder).mkdir(exist_ok=True)
    filename = url.split('/')[-1]
    dest = os.path.join(dest_folder, filename)
    try:
        with requests.get(url, stream=True, timeout=30) as r:
            r.raise_for_status()
        with open(dest, 'wb') as f:
            for chunk in r.iter_content(chunk_size=8192):
                f.write(chunk)
        return url, True, dest
    except Exception as e:
        return url, False, str(e)

urls = [
    'https://httpbin.org/bytes/1024',
    'https://httpbin.org/bytes/2048',
    'https://httpbin.org/bytes/4096',
]

# Concurrent downloads
with ThreadPoolExecutor(max_workers=5) as executor:
    futures = [executor.submit(download_file, url) for url in urls]
    for future in as_completed(futures):
        url, success, info = future.result()
        status = 'OK' if success else 'FAIL'
        print(f'[{status}] {url.split("/")[-1]} -> {info}')

```

Q36. How do you work with REST APIs using pagination?

Handle API pagination by looping through pages using cursor, offset, or link headers.

```

import requests

def fetch_all_pages(base_url, params=None, page_param='page',
                    per_page=100):
    """Fetch all pages from a paginated REST API."""
    params = params or {}
    params[page_param] = per_page
    all_items = []
    page = 1

    while True:
        params[page_param] = page
        resp = requests.get(base_url, params=params)
        resp.raise_for_status()
        data = resp.json()

```

```

# GitHub-style: list response
if isinstance(data, list):
    if not data:
        break
    all_items.extend(data)
    page += 1
# Standard: items in 'results' key
elif isinstance(data, dict):
    items = data.get('results', data.get('items', []))
    all_items.extend(items)
    if not data.get('next'): # No next page link
        break
    page += 1

return all_items

# Cursor-based pagination
def fetch_cursor_paginated(url, token=None):
    all_data = []
    while url:
        resp = requests.get(url, params={'cursor': token} if token else {})
        data = resp.json()
        all_data.extend(data.get('items', []))
        token = data.get('next_cursor')
        url = data.get('next_url')
    return all_data

```

Q37. How do you cache HTTP responses to avoid redundant requests?

Use requests-cache for transparent caching or implement a simple file-based cache with pickle.

```

import requests
import hashlib
import json
import os
import time
from pathlib import Path

class FileCache:
    def __init__(self, cache_dir='.cache', ttl=3600):
        self.cache_dir = Path(cache_dir)
        self.cache_dir.mkdir(exist_ok=True)
        self.ttl = ttl # seconds

    def _key(self, url):
        return hashlib.md5(url.encode()).hexdigest()

    def get(self, url):
        path = self.cache_dir / self._key(url)

```

```

if path.exists():
    data = json.loads(path.read_text())
    if time.time() - data['timestamp'] < self.ttl:
        return data['content']
    path.unlink() # Expired
    return None

def set(self, url, content):
    path = self.cache_dir / self._key(url)
    path.write_text(json.dumps({
        'url': url,
        'content': content,
        'timestamp': time.time()
    }))

cache = FileCache(ttl=3600)

def cached_get(url):
    if cached := cache.get(url):
        print(f'Cache HIT: {url}')
        return cached
    print(f'Cache MISS: {url}')
    resp = requests.get(url)
    resp.raise_for_status()
    data = resp.json()
    cache.set(url, data)
    return data

```

Q38. How do you handle authentication in API requests?

Different APIs use different auth methods: Basic, Bearer tokens, API keys, and OAuth.

```

import requests
from requests.auth import HTTPBasicAuth, HTTPDigestAuth

# Basic authentication
response = requests.get(
    'https://api.example.com/data',
    auth=HTTPBasicAuth('username', 'password')
    # or shorthand: auth=('username', 'password')
)

# Bearer token (OAuth2, JWT)
headers = {'Authorization': 'Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...'}
response = requests.get('https://api.example.com/protected', headers=headers)

# API Key in header
response = requests.get(
    'https://api.service.com/endpoint',
    headers={'X-API-Key': 'your-api-key-here'}
)

```

```
)

# API Key in query param
response = requests.get(
    'https://api.service.com/endpoint',
    params={'api_key': 'your-api-key-here'}
)

# Session with auth
session = requests.Session()
session.headers.update({'Authorization': 'Bearer mytoken'})
r1 = session.get('https://api.example.com/users')
r2 = session.get('https://api.example.com/posts')

# OAuth2 client credentials flow
def get_oauth_token(client_id, client_secret, token_url):
    resp = requests.post(token_url, data={
        'grant_type': 'client_credentials',
        'client_id': client_id,
        'client_secret': client_secret,
    })
    return resp.json()['access_token']
```

Section 5: Scheduling & Task Automation (Q39–Q45)

Q39. How do you schedule recurring tasks in Python?

Use the schedule library for simple cron-like task scheduling within a Python script.

```
import schedule
import time
import logging
from datetime import datetime

logging.basicConfig(level=logging.INFO,
format='%(asctime)s %(message)s')

def backup_database():
    logging.info('Running database backup...')
    # your backup logic here

def send_daily_report():
    logging.info('Sending daily report...')

def cleanup_temp_files():
    logging.info('Cleaning up temp files...')

# Schedule tasks
schedule.every(10).minutes.do(cleanup_temp_files)
schedule.every().hour.do(backup_database)
schedule.every().day.at('09:00').do(send_daily_report)
schedule.every().monday.at('08:00').do(send_daily_report)
schedule.every(2).to(5).hours.do(backup_database) # random interval

# Tag tasks for cancellation
schedule.every().day.do(cleanup_temp_files).tag('cleanup')
schedule.cancel_job(schedule.get_jobs('cleanup')[0])

# Run in a loop
while True:
    schedule.run_pending()
    time.sleep(1)
```

Q40. How do you create a cron job from Python?

Use the python-crontab library to programmatically manage system cron jobs.

```
from crontab import CronTab
import os

# Access user's crontab
cron = CronTab(user=True) # current user
# or: cron = CronTab(user='specific_user')
```



```

# Create a new job
job = cron.new(
    command='/usr/bin/python3 /home/user/scripts/backup.py',
    comment='Daily backup script'
)

# Set schedule
job.minute.on(0) # at minute 0
job.hour.on(2) # at 2 AM
job.day.every(1) # every day

# Or use cron string
job.setall('0 2 * * *') # 2:00 AM every day

# Enable/disable
job.enable()
job.enable(False) # disable

# Write to crontab
cron.write()
print('Cron job created!')

# List all cron jobs
for j in cron:
    print(j)

# Remove job
cron.remove(job)
cron.write()

# Remove all jobs with comment
cron.remove_all(comment='Daily backup script')
cron.write()

```

Q41. How do you use APScheduler for advanced scheduling?

APScheduler provides cron, interval, and one-off scheduling with persistence and job management.

```

from apscheduler.schedulers.blocking import BlockingScheduler
from apscheduler.schedulers.background import BackgroundScheduler
from apscheduler.triggers.cron import CronTrigger
from datetime import datetime

def my_job(name='world'):
    print(f'Hello {name}! Time: {datetime.now()}')

# Background scheduler (non-blocking)
scheduler = BackgroundScheduler()

# Interval trigger

```

```

scheduler.add_job(my_job, 'interval',
minutes=30,
id='interval_job',
kwargs={'name': 'Alice'})

# Cron trigger
scheduler.add_job(my_job, CronTrigger(
day_of_week='mon-fri',
hour=9,
minute=0
))

# One-time job
from datetime import datetime, timedelta
run_at = datetime.now() + timedelta(seconds=30)
scheduler.add_job(my_job, 'date', run_date=run_at)

scheduler.start()

# Manage jobs
scheduler.pause_job('interval_job')
scheduler.resume_job('interval_job')
scheduler.remove_job('interval_job')
scheduler.print_jobs()

import time
time.sleep(60)
scheduler.shutdown()

```

Q42. How do you implement a simple task queue?

Use Python's built-in queue.Queue for thread-safe task queues, or use Celery for distributed task queues.

```

import queue
import threading
import time
import logging

logging.basicConfig(level=logging.INFO)

def worker(q, worker_id):
    while True:
        try:
            task = q.get(timeout=5)
            if task is None: # Poison pill to stop worker
                break
            logging.info(f'Worker {worker_id} processing: {task}')
            time.sleep(0.5) # Simulate work
            q.task_done()
        except queue.Empty:

```

```

break

# Create task queue
task_queue = queue.Queue(maxsize=100)

# Start worker threads
num_workers = 3
threads = []
for i in range(num_workers):
    t = threading.Thread(target=worker, args=(task_queue, i+1))
    t.daemon = True
    t.start()
    threads.append(t)

# Add tasks
for i in range(10):
    task_queue.put(f'task_{i}')

# Wait for all tasks to be processed
task_queue.join()
print('All tasks completed!')

# Stop workers
for _ in range(num_workers):
    task_queue.put(None) # Send poison pills
for t in threads:
    t.join()

```

Q43. How do you implement retry logic with exponential backoff?

Implement retry with increasing delays to handle transient failures gracefully.

```

import time
import random
import functools
import logging

def retry(max_attempts=3, base_delay=1, max_delay=60,
exceptions=(Exception,), backoff=2):
    """Retry decorator with exponential backoff."""
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            delay = base_delay
            for attempt in range(1, max_attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    if attempt == max_attempts:
                        logging.error(f'All {max_attempts} attempts failed')

```

```

raise
# Add jitter to prevent thundering herd
jitter = random.uniform(0, delay * 0.1)
wait = min(delay + jitter, max_delay)
logging.warning(
    f'Attempt {attempt}/{max_attempts} failed: {e}. '
    f'Retrying in {wait:.1f}s...'
)
time.sleep(wait)
delay = min(delay * backoff, max_delay)
return None
return wrapper
return decorator

@retry(max_attempts=4, base_delay=2, exceptions=(ConnectionError, TimeoutError))
def fetch_data(url):
    import requests
    return requests.get(url, timeout=5).json()

result = fetch_data('https://api.example.com/data')

```

Q44. How do you run a Python script as a daemon/background service?

Use systemd service units on Linux to run Python scripts as background services.

```

# systemd service file: /etc/systemd/system/myservice.service
# [Unit]
# Description=My Python Automation Service
# After=network.target
#
# [Service]
# Type=simple
# User=ubuntu
# WorkingDirectory=/home/ubuntu/myapp
# ExecStart=/home/ubuntu/myapp/venv/bin/python main.py
# Restart=always
# RestartSec=10
# StandardOutput=journal
# StandardError=journal
#
# [Install]
# WantedBy=multi-user.target

# Python daemon using unix double-fork
import os
import sys

def daemonize():
    # First fork
    try:

```

```

pid = os.fork()
if pid > 0:
    sys.exit(0) # Parent exits
except OSError as e:
    sys.exit(1)

os.setsid() # New session

# Second fork
try:
    pid = os.fork()
    if pid > 0:
        sys.exit(0)
    except OSError as e:
        sys.exit(1)

# Redirect standard streams
sys.stdout.flush()
sys.stderr.flush()
os.dup2(open(os.devnull, 'r').fileno(), sys.stdin.fileno())

def main_loop():
    import time
    while True:
        # do work
        time.sleep(60)

daemonize()
main_loop()

```

Q45. How do you implement a watchdog to monitor and restart a process?

Use subprocess to launch a process and monitor it, restarting if it crashes.

```

import subprocess
import time
import logging
import sys

logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s %(levelname)s: %(message)s')

class ProcessWatchdog:
    def __init__(self, cmd, restart_delay=5, max_restarts=10):
        self.cmd = cmd
        self.restart_delay = restart_delay
        self.max_restarts = max_restarts
        self.restart_count = 0

    def run(self):

```

```
while self.restart_count <= self.max_restarts:
    logging.info(f'Starting process: {" ".join(self.cmd)}')
    proc = subprocess.Popen(self.cmd)
    exit_code = proc.wait()

    if exit_code == 0:
        logging.info('Process exited cleanly.')
        break

    self.restart_count += 1
    logging.warning(
        f'Process exited with code {exit_code}. '
        f'Restart {self.restart_count}/{self.max_restarts}'
    )

    if self.restart_count > self.max_restarts:
        logging.error('Max restarts reached. Giving up.')
        sys.exit(1)

    time.sleep(self.restart_delay)

watchdog = ProcessWatchdog(
    cmd=['python3', 'myapp.py'],
    restart_delay=5,
    max_restarts=10
)
watchdog.run()
```

Section 6: Concurrency & Parallelism (Q46–Q54)

Q46. What is the difference between threading and multiprocessing in Python?

Threading shares memory but is limited by the GIL for CPU tasks. Multiprocessing bypasses the GIL with separate processes.

```
import threading
import multiprocessing
import time

def cpu_task(n):
    return sum(i * i for i in range(n))

def io_task(url):
    import urllib.request
    with urllib.request.urlopen(url) as r:
        return len(r.read())

# Threading - best for I/O-bound tasks
threads = [threading.Thread(target=cpu_task, args=(10**6,))
            for _ in range(4)]
t0 = time.time()
for t in threads: t.start()
for t in threads: t.join()
print(f'Threading: {time.time()-t0:.2f}s')

# Multiprocessing - best for CPU-bound tasks
procs = [multiprocessing.Process(target=cpu_task, args=(10**6,))
          for _ in range(4)]
t0 = time.time()
for p in procs: p.start()
for p in procs: p.join()
print(f'Multiprocessing: {time.time()-t0:.2f}s')

# Key point: GIL allows only one thread to execute Python bytecode
# at a time, so threading doesn't speed up CPU-bound tasks.
```

Q47. How do you use ThreadPoolExecutor and ProcessPoolExecutor?

The concurrent.futures module provides high-level interfaces for both thread and process pools.

```
from concurrent.futures import (ThreadPoolExecutor, ProcessPoolExecutor,
                                as_completed)
import requests

urls = [f'https://httpbin.org/delay/{i%3}' for i in range(6)]

# Thread pool for I/O-bound work
def fetch(url):
    resp = requests.get(url, timeout=10)
```

```

return url, resp.status_code

with ThreadPoolExecutor(max_workers=4) as executor:
    futures = {executor.submit(fetch, url): url for url in urls}
    for future in as_completed(futures):
        url = futures[future]
        try:
            _, code = future.result()
            print(f'{url}: {code}')
        except Exception as e:
            print(f'{url} failed: {e}')

# Process pool for CPU-bound work
def compute(n):
    return sum(i**2 for i in range(n))

numbers = [10**6, 2*10**6, 3*10**6, 4*10**6]
with ProcessPoolExecutor(max_workers=4) as executor:
    results = list(executor.map(compute, numbers))
    print(results)

```

Q48. How do you use asyncio for async I/O operations?

asyncio enables non-blocking I/O using coroutines, perfect for concurrent network or file operations.

```

import asyncio
import aiohttp # pip install aiohttp
import time

async def fetch(session, url):
    async with session.get(url) as resp:
        return url, resp.status, len(await resp.read())

async def main():
    urls = [
        'https://httpbin.org/get',
        'https://httpbin.org/ip',
        'https://httpbin.org/headers',
    ]
    async with aiohttp.ClientSession() as session:
        tasks = [fetch(session, url) for url in urls]
        results = await asyncio.gather(*tasks, return_exceptions=True)
        for r in results:
            if isinstance(r, Exception):
                print(f'Error: {r}')
            else:
                url, status, size = r
                print(f'{status} {size:6d}B {url}')

# asyncio.run() is the entry point

```



```

asyncio.run(main())

# Async with timeout
async def fetch_with_timeout(url, timeout=5):
    try:
        async with asyncio.timeout(timeout):
            async with aiohttp.ClientSession() as s:
                async with s.get(url) as r:
                    return await r.json()
    except asyncio.TimeoutError:
        return None

```

Q49. How do you use asyncio queues for producer-consumer patterns?

asyncio.Queue enables clean producer-consumer workflows in async code without locks.

```

import asyncio
import random

async def producer(queue, n_items):
    for i in range(n_items):
        item = f'item_{i}'
        await queue.put(item)
        print(f'Produced: {item}')
        await asyncio.sleep(random.uniform(0.1, 0.5))
    await queue.put(None) # Signal done

async def consumer(queue, worker_id):
    while True:
        item = await queue.get()
        if item is None:
            await queue.put(None) # Pass signal to next worker
            break
        print(f'Worker {worker_id} processing: {item}')
        await asyncio.sleep(random.uniform(0.2, 0.8))
        queue.task_done()

async def main():
    queue = asyncio.Queue(maxsize=5)
    n_workers = 3

    producer_task = asyncio.create_task(producer(queue, 10))
    workers = [asyncio.create_task(consumer(queue, i))
               for i in range(n_workers)]

    await producer_task
    await asyncio.gather(*workers)
    print('All done!')

asyncio.run(main())

```

Q50. How do you use threading.Lock to prevent race conditions?

Use threading.Lock or threading.RLock to ensure only one thread accesses shared resources at a time.

```
import threading
import time

# Without Lock - race condition
counter = 0
def increment_unsafe():
    global counter
    for _ in range(100000):
        counter += 1 # Not thread-safe!

# With Lock - thread-safe
lock = threading.Lock()
safe_counter = 0
def increment_safe():
    global safe_counter
    for _ in range(100000):
        with lock:
            safe_counter += 1

threads = [threading.Thread(target=increment_safe) for _ in range(5)]
for t in threads: t.start()
for t in threads: t.join()
print(f'Safe counter: {safe_counter}') # Always 500000

# threading.RLock (reentrant - same thread can acquire multiple times)
rlock = threading.RLock()
def recursive_task(n):
    with rlock:
        if n > 0:
            recursive_task(n - 1)

# Thread-local storage (each thread has its own copy)
local = threading.local()
def worker():
    local.value = threading.get_ident()
    time.sleep(0.1)
    print(f'Thread {threading.get_ident()}: {local.value}')
```

Q51. How do you use multiprocessing.Pool for parallel computation?

Pool provides map/apply methods for distributing work across multiple processes.

```
from multiprocessing import Pool, cpu_count
import os

def process_chunk(chunk):
    """CPU-intensive task per data chunk."""
    return sum(x**2 for x in chunk)
```

```

def worker_initializer():
    print(f'Worker PID: {os.getpid()} started')

# Create a list of chunks
data = list(range(1000000))
chunk_size = len(data) // cpu_count()
chunks = [data[i:i+chunk_size]
           for i in range(0, len(data), chunk_size)]

# Pool.map() - blocks until all done
with Pool(processes=cpu_count(),
          initializer=worker_initializer) as pool:
    results = pool.map(process_chunk, chunks)
    total = sum(results)
    print(f'Total: {total}')

# Pool.imap() - lazy iterator (memory efficient)
with Pool(processes=4) as pool:
    for result in pool.imap(process_chunk, chunks, chunksize=1):
        print(f'Got: {result}')

# Pool.starmap() - multiple arguments
def add(a, b):
    return a + b

with Pool() as pool:
    results = pool.starmap(add, [(1, 2), (3, 4), (5, 6)])
    print(results) # [3, 7, 11]

```

Q52. How do you share data between processes?

Use multiprocessing.Value, Array, Manager, or Queue for inter-process communication.

```

from multiprocessing import Process, Value, Array, Manager, Queue
import ctypes

# Shared Value (single value)
counter = Value(ctypes.c_int, 0)

def increment(shared_val, n):
    for _ in range(n):
        with shared_val.get_lock():
            shared_val.value += 1

procs = [Process(target=increment, args=(counter, 1000)) for _ in range(4)]
for p in procs: p.start()
for p in procs: p.join()
print(f'Counter: {counter.value}') # 4000

```

```

# Shared Array
arr = Array(ctypes.c_double, range(10))
print(list(arr[:]))

# Manager - supports dicts, lists, etc.
with Manager() as manager:
    shared_dict = manager.dict()
    shared_list = manager.list()

def worker(d, l, key):
    d[key] = key * 2
    l.append(key)

procs = [Process(target=worker, args=(shared_dict, shared_list, i))
          for i in range(5)]
for p in procs: p.start()
for p in procs: p.join()
print(dict(shared_dict))
print(list(shared_list))

```

Q53. What is `asyncio.gather()` vs `asyncio.wait()`?

`gather()` runs tasks concurrently and returns results in order. `wait()` provides more control over completion conditions.

```

import asyncio

async def task(n, delay):
    await asyncio.sleep(delay)
    if n == 2:
        raise ValueError(f'Task {n} failed!')
    return f'result_{n}'

# asyncio.gather() - all run concurrently, results in order
async def demo_gather():
    # return_exceptions=True prevents one failure from canceling others
    results = await asyncio.gather(
        task(1, 0.1),
        task(2, 0.2),
        task(3, 0.3),
        return_exceptions=True
    )
    for r in results:
        if isinstance(r, Exception):
            print(f'Error: {r}')
        else:
            print(f'Success: {r}')

# asyncio.wait() - more control
async def demo_wait():

```

```

tasks = [asyncio.create_task(task(i, i*0.1)) for i in range(4)]
done, pending = await asyncio.wait(
    tasks,
    return_when=asyncio.FIRST_COMPLETED # or ALL_COMPLETED
)
for t in done:
    try:
        print(f'Done: {t.result()}')
    except Exception as e:
        print(f'Failed: {e}')
    for t in pending:
        t.cancel() # Cancel remaining

asyncio.run(demo_gather())

```

Q54. How do you use a semaphore to limit concurrent tasks?

`asyncio.Semaphore` and `threading.Semaphore` limit the number of concurrent operations to prevent overloading.

```

import asyncio
import aiohttp

async def fetch(session, sem, url):
    async with sem: # Limit concurrency
        async with session.get(url) as resp:
            return await resp.text()

async def main():
    # Max 5 concurrent requests
    sem = asyncio.Semaphore(5)
    urls = [f'https://httpbin.org/get?n={i}' for i in range(20)]

    async with aiohttp.ClientSession() as session:
        tasks = [fetch(session, sem, url) for url in urls]
        results = await asyncio.gather(*tasks, return_exceptions=True)
        print(f'Fetched {len(results)} pages')

    asyncio.run(main())

# Threading semaphore
import threading
import time

sem = threading.Semaphore(3) # Max 3 concurrent

def worker(n):
    with sem:
        print(f'Worker {n} running')
        time.sleep(1)

```

```
print(f'Worker {n} done')
```

```
threads = [threading.Thread(target=worker, args=(i,)) for i in range(8)]  
for t in threads: t.start()  
for t in threads: t.join()
```

Section 7: Logging & Error Handling (Q55–Q63)

Q55. How do you set up Python logging properly?

Use the logging module with appropriate handlers, formatters, and levels for production-grade logging.

```
import logging
import logging.handlers
import sys

def setup_logger(name, log_file=None, level=logging.INFO):
    logger = logging.getLogger(name)
    logger.setLevel(level)

    formatter = logging.Formatter(
        '%(asctime)s - %(name)s - %(levelname)s - %(message)s',
        datefmt='%Y-%m-%d %H:%M:%S'
    )

    # Console handler
    console = logging.StreamHandler(sys.stdout)
    console.setLevel(level)
    console.setFormatter(formatter)
    logger.addHandler(console)

    # File handler (optional)
    if log_file:
        # Rotating file: max 10MB, keep 5 backups
        file_handler = logging.handlers.RotatingFileHandler(
            log_file, maxBytes=10*1024*1024, backupCount=5
        )
        file_handler.setLevel(level)
        file_handler.setFormatter(formatter)
        logger.addHandler(file_handler)

    return logger

logger = setup_logger('myapp', 'app.log', logging.DEBUG)

logger.debug('Debug message')
logger.info('Info message')
logger.warning('Warning!')
logger.error('Error occurred')
logger.critical('Critical failure')

# Structured logging with extra context
logger.info('User logged in', extra={'user_id': 42, 'ip': '127.0.0.1'})
```

Q56. How do you log exceptions with full tracebacks?

Use `logger.exception()` inside `except` blocks to automatically log the full traceback.

```
import logging
import traceback

logging.basicConfig(level=logging.DEBUG)
logger = logging.getLogger(__name__)

# Method 1: logger.exception() - includes traceback automatically
def divide(a, b):
    try:
        return a / b
    except ZeroDivisionError:
        logger.exception('Division by zero error')
    return None

divide(10, 0) # logs error + full traceback

# Method 2: logger.error with exc_info=True
def process_data(data):
    try:
        return int(data) * 2
    except ValueError as e:
        logger.error('Invalid data: %s', data, exc_info=True)
    return None

# Method 3: Manual traceback
def risky_operation():
    try:
        raise RuntimeError('Something went wrong')
    except RuntimeError as e:
        tb = traceback.format_exc()
        logger.error('Operation failed:\n%s', tb)

# Context manager approach
from contextlib import contextmanager

@contextmanager
def log_errors(operation_name):
    try:
        yield
    except Exception as e:
        logger.exception(f'{operation_name} failed')
        raise

with log_errors('database_write'):
    raise ConnectionError('DB is down')
```

Q57. How do you create custom exception classes?

Inherit from `Exception` or specific built-in exceptions to create meaningful custom exceptions.


```

class AppError(Exception):
    """Base exception for this application."""
    def __init__(self, message, error_code=None):
        super().__init__(message)
        self.error_code = error_code

    def __str__(self):
        if self.error_code:
            return f'[{self.error_code}] {super().__str__()}'
        return super().__str__()

class DatabaseError(AppError):
    """Raised when database operations fail."""
    pass

class ValidationError(AppError):
    """Raised for invalid input data."""
    def __init__(self, field, message):
        super().__init__(f'Validation failed for {field}: {message}')
        self.field = field

class NetworkError(AppError):
    """Raised for network-related failures."""
    def __init__(self, url, status_code):
        super().__init__(f'HTTP {status_code} for {url}')
        self.url = url
        self.status_code = status_code

# Usage
try:
    raise ValidationError('email', 'invalid format')
except ValidationError as e:
    print(f'Field: {e.field}')
    print(str(e))
except AppError as e:
    print(f'App error: {e}')

```

Q58. What is the difference between finally, else, and except in try blocks?

finally always runs. else runs only if no exception occurred. except handles specific exceptions.

```

# Full try/except/else/finally structure
def read_config(path):
    f = None
    try:
        f = open(path, 'r')
        config = f.read()
        result = eval(config) # might raise SyntaxError
    except FileNotFoundError:
        print('Config file not found')
    finally:
        return {}

```

```

except (SyntaxError, ValueError) as e:
    print(f'Config parse error: {e}')
    return {}
except Exception as e:
    print(f'Unexpected error: {e}')
    raise # Re-raise unknown errors
else:
    # Only runs if NO exception was raised in try block
    print('Config loaded successfully')
    return result
finally:
    # ALWAYS runs - cleanup goes here
    if f:
        f.close()
    print('File closed')

# Chained exceptions
try:
    try:
        int('abc')
    except ValueError as e:
        raise RuntimeError('Processing failed') from e
except RuntimeError as e:
    print(e)
    print(f'Caused by: {e.__cause__}')

```

Q59. How do you use context managers for resource management?

Implement `__enter__` and `__exit__` or use `@contextmanager` for safe resource management.

```

from contextlib import contextmanager
import time

# Class-based context manager
class Timer:
    def __enter__(self):
        self.start = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.elapsed = time.time() - self.start
        print(f'Elapsed: {self.elapsed:.3f}s')
        return False # Don't suppress exceptions

with Timer() as t:
    time.sleep(0.1)
    print(f'Took {t.elapsed:.3f}s')

# Generator-based context manager
@contextmanager

```

```

def managed_connection(host, port):
    conn = None
    try:
        print(f'Connecting to {host}:{port}')
        conn = {'host': host, 'port': port, 'open': True} # Simulated
        yield conn
    except Exception as e:
        print(f'Connection error: {e}')
        raise
    finally:
        if conn:
            conn['open'] = False
            print('Connection closed')

with managed_connection('localhost', 5432) as conn:
    print(f'Using connection: {conn}')

# Suppress specific exceptions
from contextlib import suppress
with suppress(FileNotFoundError):
    open('nonexistent.txt') # No error raised

```

Q60. How do you implement structured error handling in a script?

Use a layered approach: validate inputs, handle specific errors, log with context, and provide useful exit codes.

```

import sys
import logging
from pathlib import Path

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

class ScriptError(Exception):
    def __init__(self, message, exit_code=1):
        super().__init__(message)
        self.exit_code = exit_code

def validate_args(args):
    if not args:
        raise ScriptError('No arguments provided. Usage: script.py <file>', 2)
    path = Path(args[0])
    if not path.exists():
        raise ScriptError(f'File not found: {path}', 3)
    if not path.is_file():
        raise ScriptError(f'Not a file: {path}', 4)
    return path

def process_file(path):
    try:

```

```

data = path.read_text(encoding='utf-8')
return data.upper()
except PermissionError:
    raise ScriptError(f'No permission to read: {path}', 5)
except UnicodeDecodeError:
    raise ScriptError(f'Encoding error in: {path}', 6)

def main(args):
    path = validate_args(args)
    logger.info(f'Processing: {path}')
    result = process_file(path)
    print(result[:100])

if __name__ == '__main__':
    try:
        main(sys.argv[1:])
    except ScriptError as e:
        logger.error(str(e))
        sys.exit(e.exit_code)
    except KeyboardInterrupt:
        logger.info('Interrupted by user')
        sys.exit(130)

```

Q61. How do you use Python's warnings module?

Use the warnings module to issue deprecation warnings, user warnings, and filter warnings.

```

import warnings

# Issue a warning
def old_function(x):
    warnings.warn(
        'old_function is deprecated, use new_function instead.',
        DeprecationWarning,
        stacklevel=2 # Show caller's location
    )
    return x * 2

old_function(5) # Triggers DeprecationWarning

# Warning categories
warnings.warn('Low disk space', ResourceWarning)
warnings.warn('Configuration missing', UserWarning)

# Filter warnings
warnings.filterwarnings('ignore', category=DeprecationWarning)
warnings.filterwarnings('error', category=RuntimeWarning)
warnings.filterwarnings('once', category=UserWarning)

# Context manager for temporary warning suppression

```

```

with warnings.catch_warnings():
    warnings.simplefilter('ignore')
    old_function(10) # No warning shown

# Always show DeprecationWarning (normally hidden in production)
warnings.filterwarnings('always', category=DeprecationWarning)

# Convert warnings to errors in test environments
import os
if os.environ.get('TESTING'):
    warnings.filterwarnings('error')

```

Q62. How do you use assert statements for debugging?

assert is for internal consistency checks during development. Disable with python -O. Never use for input validation.

```

# Basic assert
def divide(a, b):
    assert b != 0, f'Divisor cannot be zero, got {b}'
    return a / b

divide(10, 2) # OK
divide(10, 0) # AssertionError: Divisor cannot be zero, got 0

# Assert with complex conditions
def process(data):
    assert isinstance(data, list), f'Expected list, got {type(data)}'
    assert len(data) > 0, 'Data cannot be empty'
    assert all(isinstance(x, (int, float)) for x in data), \
        'All elements must be numeric'
    return sum(data) / len(data)

# Testing invariants
class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance

    def deposit(self, amount):
        assert amount > 0, 'Deposit must be positive'
        self.balance += amount
        assert self.balance >= 0, 'Balance invariant violated'

    def withdraw(self, amount):
        assert amount > 0, 'Withdrawal must be positive'
        assert amount <= self.balance, 'Insufficient funds'
        self.balance -= amount

# NOTE: asserts are disabled with python -O (optimize) flag
# NEVER use assert for security or input validation in production!

```

Q63. How do you write a global exception handler for unhandled exceptions?

Use sys.excepthook to catch all unhandled exceptions and log them before the program exits.

```
import sys
import logging
import traceback

logging.basicConfig(
    filename='crash.log',
    level=logging.ERROR,
    format='%(asctime)s %(levelname)s %(message)s'
)

def global_exception_handler(exc_type, exc_value, exc_traceback):
    # Don't log keyboard interrupts
    if isinstance(exc_type, KeyboardInterrupt):
        sys.__excepthook__(exc_type, exc_value, exc_traceback)
        return

    logging.critical(
        'Unhandled exception:',
        exc_info=(exc_type, exc_value, exc_traceback)
    )
    print(f'FATAL ERROR: {exc_value}', file=sys.stderr)
    print('See crash.log for details', file=sys.stderr)

# Install global handler
sys.excepthook = global_exception_handler

# For threads
import threading

def thread_exception_handler(args):
    logging.critical(
        f'Unhandled exception in thread {args.thread.name}',
        exc_info=(args.exc_type, args.exc_value, args.exc_traceback)
    )

threading.excepthook = thread_exception_handler

# For asyncio
import asyncio

def async_exception_handler(loop, context):
    exception = context.get('exception')
    logging.critical(f'Async exception: {context["message"]}',
        exc_info=exception)

loop = asyncio.new_event_loop()
loop.set_exception_handler(async_exception_handler)
```

Section 8: Config, Environment & CLI (Q64–Q72)

Q64. How do you manage environment variables and .env files?

Use `os.environ` for env vars and `python-dotenv` to load from `.env` files for local development.

```
import os
from dotenv import load_dotenv # pip install python-dotenv

# Load .env file
load_dotenv() # loads from .env in current directory
load_dotenv('/path/to/.env') # specify path

# Read environment variables
db_url = os.environ.get('DATABASE_URL')
db_url = os.getenv('DATABASE_URL', 'sqlite:///default.db') # with default

api_key = os.environ['API_KEY'] # raises KeyError if missing

# Set env vars (current process only)
os.environ['MY_VAR'] = 'my_value'

# Check if var exists
if 'DEBUG' in os.environ:
    print('Debug mode enabled')

# Get all env vars as dict
env_snapshot = dict(os.environ)

# Sample .env file content:
# DATABASE_URL=postgres://user:pass@localhost/mydb
# API_KEY=secret123
# DEBUG=true
# MAX_WORKERS=4

# Type conversion helpers
def get_bool_env(key, default=False):
    val = os.getenv(key, str(default)).lower()
    return val in ('1', 'true', 'yes', 'on')

def get_int_env(key, default=0):
    return int(os.getenv(key, default))

DEBUG = get_bool_env('DEBUG', False)
WORKERS = get_int_env('MAX_WORKERS', 4)
```

Q65. How do you parse configuration files with configparser?

`configparser` reads INI-style config files with sections and key-value pairs.

```

import configparser
from pathlib import Path

# Sample config.ini:
# [database]
# host = localhost
# port = 5432
# name = mydb
#
# [app]
# debug = true
# max_workers = 4
# allowed_hosts = localhost, 127.0.0.1

config = configparser.ConfigParser()
config.read('config.ini')

# Read values
host = config.get('database', 'host')
port = config.getint('database', 'port')
debug = config.getboolean('app', 'debug')
workers = config.getint('app', 'max_workers', fallback=2)

# Read list value
hosts = [h.strip() for h in
config.get('app', 'allowed_hosts').split(',')]

# Write config
config['new_section'] = {'key': 'value', 'enabled': 'true'}
with open('config.ini', 'w') as f:
    config.write(f)

# Check sections and options
print(config.sections()) # ['database', 'app']
print(config.has_option('app', 'debug')) # True
print(dict(config['database'])) # section as dict

```

Q66. How do you build a CLI tool with argparse?

argparse parses command-line arguments, provides help text, type validation, and subcommands.

```

import argparse
import sys

def create_parser():
    parser = argparse.ArgumentParser(
        description='File processing tool',
        formatter_class=argparse.ArgumentDefaultsHelpFormatter
    )
    parser.add_argument('input', help='Input file path')
    parser.add_argument('-o', '--output', default='output.txt',

```



```

help='Output file path')
parser.add_argument('-n', '--count', type=int, default=10,
help='Number of lines to process')
parser.add_argument('-v', '--verbose', action='store_true',
help='Enable verbose output')
parser.add_argument('--format', choices=['json', 'csv', 'txt'],
default='txt', help='Output format')
return parser

# Subcommands
def create_subcommand_parser():
    parser = argparse.ArgumentParser(description='My CLI Tool')
    subparsers = parser.add_subparsers(dest='command', required=True)

    # 'process' subcommand
    proc = subparsers.add_parser('process', help='Process files')
    proc.add_argument('file', help='File to process')

    # 'clean' subcommand
    clean = subparsers.add_parser('clean', help='Clean output')
    clean.add_argument('--dry-run', action='store_true')
    return parser

if __name__ == '__main__':
    parser = create_parser()
    args = parser.parse_args()
    if args.verbose:
        print(f'Processing {args.input}')
        print(f'Output: {args.output}, Format: {args.format}')

```

Q67. How do you build a rich CLI with Click?

Click provides decorators for building CLI tools with less boilerplate than argparse, supporting groups and options.

```

import click

@click.group()
@click.version_option('1.0.0')
@click.option('--verbose', '-v', is_flag=True,
help='Enable verbose mode')
@click.pass_context
def cli(ctx, verbose):
    ctx.ensure_object(dict)
    ctx.obj['verbose'] = verbose

@cli.command()
@click.argument('filename')
@click.option('--count', '-n', default=10, show_default=True,
help='Number of lines')

```

```

@click.option('--format', 'fmt',
              type=click.Choice(['json', 'csv', 'txt']),
              default='txt')
@click.pass_context
def process(ctx, filename, count, fmt):
    """Process a file."""
    if ctx.obj['verbose']:
        click.echo(f'Processing {filename}')
        click.echo(f'Lines: {count}, Format: {fmt}')

@cli.command()
@click.argument('path', type=click.Path(exists=True))
@click.confirmation_option(prompt='Are you sure you want to delete?')
def delete(path):
    """Delete a file."""
    click.echo(click.style(f'Deleted: {path}', fg='red'))

if __name__ == '__main__':
    cli()

```

Q68. How do you read and write YAML configuration files?

Use PyYAML or ruamel.yaml for reading/writing YAML files, which are human-readable configs.

```

import yaml # pip install pyyaml
from pathlib import Path

# Read YAML
config_text = '''
app:
  name: MyApp
  version: 1.0
  debug: false
  max_workers: 4

database:
  host: localhost
  port: 5432
  credentials:
    username: admin
    password: secret

servers:
  - name: web1
    ip: 192.168.1.10
  - name: web2
    ip: 192.168.1.11
'''

config = yaml.safe_load(config_text)

```

```

print(config['app']['name']) # MyApp
print(config['database']['port']) # 5432
print(config['servers'][0]['name']) # web1

# Write YAML
new_config = {
    'app': {'name': 'Updated', 'debug': True},
    'features': ['auth', 'api', 'webhooks']
}

with open('config.yaml', 'w') as f:
    yaml.dump(new_config, f, default_flow_style=False, sort_keys=False)

# Load from file
path = Path('config.yaml')
if path.exists():
    config = yaml.safe_load(path.read_text())

```

Q69. How do you validate configuration using Pydantic?

Pydantic models provide automatic type validation, default values, and error messages for config objects.

```

from pydantic import BaseModel, Field, validator
from pydantic_settings import BaseSettings # pydantic v2
import os

# Settings loaded from environment variables
class DatabaseSettings(BaseModel):
    host: str = 'localhost'
    port: int = Field(default=5432, ge=1, le=65535)
    name: str
    pool_size: int = 5

class AppSettings(BaseSettings):
    app_name: str = 'MyApp'
    debug: bool = False
    max_workers: int = Field(default=4, ge=1, le=100)
    database: DatabaseSettings
    allowed_hosts: list[str] = ['localhost']
    api_key: str

class Config:
    env_file = '.env'
    env_nested_delimiter = '__'
    # DB_HOST=localhost, DB_PORT=5432 -> database.host, database.port

    @validator('allowed_hosts', pre=True)
    def parse_hosts(cls, v):
        if isinstance(v, str):
            return [h.strip() for h in v.split(',')]

```

```

return v

# Usage
try:
    settings = AppSettings(
        database={'host': 'db.example.com', 'name': 'mydb'},
        api_key='secret-key-123'
    )
    print(settings.database.host) # db.example.com
    print(settings.max_workers) # 4
except Exception as e:
    print(f'Config error: {e}')

```

Q70. How do you use dataclasses for configuration objects?

dataclasses provide a clean, Pythonic way to create configuration containers with type hints and defaults.

```

from dataclasses import dataclass, field, asdict
from typing import Optional
import json

@dataclass
class DatabaseConfig:
    host: str = 'localhost'
    port: int = 5432
    name: str = 'mydb'
    pool_size: int = 5
    ssl: bool = False

    def connection_string(self):
        proto = 'postgres'
        return f'{proto}://{self.host}:{self.port}/{self.name}'

@dataclass
class AppConfig:
    app_name: str = 'MyApp'
    debug: bool = False
    max_workers: int = 4
    database: DatabaseConfig = field(default_factory=DatabaseConfig)
    allowed_hosts: list = field(default_factory=lambda: ['localhost'])
    log_level: str = 'INFO'

    @classmethod
    def from_dict(cls, data: dict):
        db_data = data.pop('database', {})
        config = cls(**data)
        config.database = DatabaseConfig(**db_data)
        return config

    def to_json(self):

```

```

return json.dumps(asdict(self), indent=2)

config = AppConfig(debug=True, max_workers=8)
print(config.database.connection_string())
print(config.to_json())

```

Q71. How do you handle command-line argument groups and mutual exclusion?

Use argparse's `add_mutually_exclusive_group()` or `add_argument_group()` to organize and constrain options.

```

import argparse

parser = argparse.ArgumentParser(description='Data processing tool')

# Argument group for organization (display only)
input_group = parser.add_argument_group('Input Options')
input_group.add_argument('--file', '-f', help='Input file')
input_group.add_argument('--url', '-u', help='Input URL')
input_group.add_argument('--stdin', action='store_true',
help='Read from stdin')

# Mutually exclusive group (only one allowed)
output_group = parser.add_mutually_exclusive_group(required=True)
output_group.add_argument('--json', action='store_true',
help='Output as JSON')
output_group.add_argument('--csv', action='store_true',
help='Output as CSV')
output_group.add_argument('--xml', action='store_true',
help='Output as XML')

# Common options
parser.add_argument('--output', '-o', default='-',
help='Output file (- for stdout)')
parser.add_argument('--verbose', '-v', action='count', default=0,
help='Increase verbosity (-v, -vv, -vvv)')

args = parser.parse_args()
verbosity = ['ERROR', 'WARNING', 'INFO', 'DEBUG'][min(args.verbose, 3)]
print(f'Verbosity level: {verbosity}')

```

Q72. How do you add color and formatting to terminal output?

Use `colorama` for cross-platform ANSI colors, or `rich` for beautiful terminal output.

```

# Using colorama (pip install colorama)
from colorama import init, Fore, Back, Style
init(autoreset=True) # Auto-reset after each print

print(Fore.RED + 'Error: Connection failed')
print(Fore.GREEN + 'Success: Data processed')
print(Fore.YELLOW + 'Warning: Low disk space')

```

```
print(Fore.CYAN + Style.BRIGHT + 'INFO: Starting up...')
print(Back.BLUE + Fore.WHITE + 'Status: RUNNING')

# Using rich (pip install rich)
from rich.console import Console
from rich.table import Table
from rich.progress import track

console = Console()
console.print('[bold red]Error:[/bold red] Something failed')
console.print('[green]Success[/green] Operation complete')

# Rich table
table = Table(title='Server Status')
table.add_column('Server', style='cyan')
table.add_column('Status', style='green')
table.add_column('Uptime')

table.add_row('web-01', 'ONLINE', '99.9%')
table.add_row('db-01', 'ONLINE', '99.5%')
table.add_row('cache-01', '[red]OFFLINE[/red]', '0%')
console.print(table)

# Progress bar
for i in track(range(100), description='Processing...'):
    import time; time.sleep(0.01)
```

Section 9: OS & System Automation (Q73–Q81)

Q73. How do you get system information using Python?

Use the platform, psutil, and os modules to gather CPU, memory, and OS information.

```
import platform
import psutil # pip install psutil
import os

# OS info
print(platform.system()) # Linux, Windows, Darwin
print(platform.release()) # kernel version
print(platform.machine()) # x86_64
print(platform.processor()) # CPU name
print(platform.python_version()) # 3.11.0

# CPU info
print(f'CPU cores: {psutil.cpu_count(logical=False)}')
print(f'Logical CPUs: {psutil.cpu_count(logical=True)}')
print(f'CPU usage: {psutil.cpu_percent(interval=1)}%')
per_cpu = psutil.cpu_percent(interval=1, percpu=True)
print(f'Per CPU: {per_cpu}')

# Memory info
mem = psutil.virtual_memory()
print(f'Total RAM: {mem.total / 1024**3:.1f} GB')
print(f'Available: {mem.available / 1024**3:.1f} GB')
print(f'Used: {mem.percent}%')

# Disk info
for disk in psutil.disk_partitions():
    try:
        usage = psutil.disk_usage(disk.mountpoint)
        print(f'{disk.mountpoint}: {usage.percent}% used')
    except PermissionError:
        pass
```

Q74. How do you monitor running processes with psutil?

psutil provides APIs to enumerate, inspect, and manage system processes.

```
import psutil
import os

# List all processes
for proc in psutil.process_iter(['pid', 'name', 'cpu_percent']):
    try:
        info = proc.info
        if info['cpu_percent'] > 5:
            print(f'{info["pid"]:6d} {info["name"]:<20} {info["cpu_percent"]}%' )
```

```

except psutil.NoSuchProcess:
    pass

# Get process by PID
try:
    proc = psutil.Process(os.getpid())
    print(f'Name: {proc.name()}')
    print(f'Status: {proc.status()}')
    print(f'CPU: {proc.cpu_percent(interval=1)}%')
    print(f'Memory: {proc.memory_info().rss / 1024**2:.1f} MB')
    print(f'Threads: {proc.num_threads()}')
    print(f'Open files: {len(proc.open_files())}')
except psutil.NoSuchProcess:
    pass

# Find processes by name
def find_process(name):
    for proc in psutil.process_iter(['pid', 'name', 'cmdline']):
        try:
            if name.lower() in proc.info['name'].lower():
                return proc
        except (psutil.NoSuchProcess, psutil.AccessDenied):
            pass
    return None

nginx = find_process('nginx')
if nginx:
    print(f'nginx PID: {nginx.pid}')

```

Q75. How do you monitor network connections and bandwidth?

psutil provides network I/O counters and connection information for monitoring.

```

import psutil
import time

# Network interface info
stats = psutil.net_if_stats()
addrs = psutil.net_if_addrs()

for iface, stat in stats.items():
    addr_list = addrs.get(iface, [])
    ips = [a.address for a in addr_list]
    if a.family.name in ('AF_INET', 'AF_INET6')]
    print(f'{iface}: UP={stat.isup}, IPs={ips}')

# Network I/O counters
io1 = psutil.net_io_counters()
time.sleep(1)
io2 = psutil.net_io_counters()

```



```

sent_rate = (io2.bytes_sent - io1.bytes_sent) / 1024 # KB/s
recv_rate = (io2.bytes_recv - io1.bytes_recv) / 1024 # KB/s
print(f'Upload: {sent_rate:.1f} KB/s, Download: {recv_rate:.1f} KB/s')

# Active connections
connections = psutil.net_connections()
listening = [c for c in connections if c.status == 'LISTEN']
for conn in listening:
    print(f'Listening on port {conn.laddr.port}')

# Per-interface counters
per_nic = psutil.net_io_counters(pernic=True)
for nic, counters in per_nic.items():
    print(f'{nic}: sent={counters.bytes_sent}, recv={counters.bytes_recv}')

```

Q76. How do you automate SSH connections with Paramiko?

Paramiko enables SSH connections, remote command execution, and SFTP file transfers from Python.

```

import paramiko
import os

def ssh_connect(host, username, key_path=None, password=None):
    client = paramiko.SSHClient()
    client.set_missing_host_key_policy(paramiko.AutoAddPolicy())

    if key_path:
        client.connect(host, username=username,
                       key_filename=key_path)
    else:
        client.connect(host, username=username, password=password)
    return client

# Execute remote command
client = ssh_connect('myserver.com', 'ubuntu',
                    key_path='~/.ssh/id_rsa')
try:
    stdin, stdout, stderr = client.exec_command('df -h')
    output = stdout.read().decode()
    errors = stderr.read().decode()
    print(output)

# Check exit status
exit_code = stdout.channel.recv_exit_status()
print(f'Exit code: {exit_code}')
finally:
    client.close()

# SFTP file transfer

```

```

transport = paramiko.Transport(('myserver.com', 22))
transport.connect(username='ubuntu',
pkey=paramiko.RSAKey.from_private_key_file(
os.path.expanduser('~/.ssh/id_rsa')))
sftp = paramiko.SFTPClient.from_transport(transport)
sftp.put('local_file.txt', '/remote/path/file.txt') # upload
sftp.get('/remote/data.csv', 'local_data.csv') # download
sftp.close()
transport.close()

```

Q77. How do you work with the Windows registry using Python?

Use the winreg module (Windows only) to read, write, and manage registry keys.

```

import winreg # Windows only
import sys

if sys.platform != 'win32':
    print('Windows only - skipping')
else:
    # Read a registry value
    key_path = r'SOFTWARE\Microsoft\Windows\CurrentVersion'
    with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE, key_path) as key:
        value, reg_type = winreg.QueryValueEx(key, 'ProgramFilesDir')
        print(f'Program Files: {value}')

    # Create/write a registry key
    key_path = r'SOFTWARE\MyApplication'
    with winreg.CreateKey(winreg.HKEY_CURRENT_USER, key_path) as key:
        winreg.SetValueEx(key, 'Version', 0, winreg.REG_SZ, '1.0.0')
        winreg.SetValueEx(key, 'MaxWorkers', 0, winreg.REG_DWORD, 4)

    # Read back
    with winreg.OpenKey(winreg.HKEY_CURRENT_USER, key_path) as key:
        ver, _ = winreg.QueryValueEx(key, 'Version')
        print(f'Version: {ver}')

    # Enumerate subkeys
    with winreg.OpenKey(winreg.HKEY_LOCAL_MACHINE,
r'SOFTWARE') as key:
        i = 0
        while True:
            try:
                subkey = winreg.EnumKey(key, i)
                print(subkey)
                i += 1
            except OSError:
                break

```

Q78. How do you automate GUI applications with PyAutoGUI?

PyAutoGUI controls mouse and keyboard to automate desktop GUI interactions.

```
import pyautogui
import time

# Safety: pause between actions
pyautogui.PAUSE = 0.5
pyautogui.FAILSAFE = True # Move to corner to abort

# Mouse operations
screen_w, screen_h = pyautogui.size()
x, y = pyautogui.position() # Current position

pyautogui.moveTo(100, 200, duration=0.5) # Move to position
pyautogui.click(100, 200) # Left click
pyautogui.doubleClick(100, 200) # Double click
pyautogui.rightClick(100, 200) # Right click
pyautogui.dragTo(500, 300, duration=1.0) # Drag

# Keyboard operations
pyautogui.write('Hello, World!', interval=0.05) # Type text
pyautogui.press('enter')
pyautogui.hotkey('ctrl', 'c') # Ctrl+C
pyautogui.hotkey('alt', 'f4') # Alt+F4

# Screenshot and image recognition
screenshot = pyautogui.screenshot()
screenshot.save('screen.png')

# Find image on screen
location = pyautogui.locateOnScreen('button.png', confidence=0.9)
if location:
    pyautogui.click(location)
    print(f'Found button at {location}')
else:
    print('Button not found')
```

Q79. How do you send emails programmatically using Python?

Use `smtplib` and `email` modules for sending plain text and HTML emails with attachments.

```
import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.mime.base import MIMEBase
from email import encoders
import os

def send_email(to, subject, body, attachments=None,
               smtp_host='smtp.gmail.com', smtp_port=587,
               username=None, password=None):
    msg = MIMEMultipart()
```

```

msg['From'] = username
msg['To'] = ', '.join(to) if isinstance(to, list) else to
msg['Subject'] = subject

# HTML body with plain text fallback
msg.attach(MIMEText(body, 'html'))

# Attachments
for filepath in (attachments or []):
    with open(filepath, 'rb') as f:
        part = MIMEBase('application', 'octet-stream')
        part.set_payload(f.read())
        encoders.encode_base64(part)
        part.add_header('Content-Disposition',
            f'attachment; filename={os.path.basename(filepath)}')
        msg.attach(part)

with smtplib.SMTP(smtp_host, smtp_port) as server:
    server.ehlo()
    server.starttls()
    server.login(username, password)
    server.sendmail(username, to, msg.as_string())
    print('Email sent successfully!')

send_email(
    to='recipient@example.com',
    subject='Automation Report',
    body='<h1>Report</h1><p>Process completed.</p>',
    username=os.getenv('EMAIL_USER'),
    password=os.getenv('EMAIL_PASS')
)

```

Q80. How do you interact with databases using Python?

Use sqlite3 (built-in) or SQLAlchemy for database-agnostic ORM operations.

```

import sqlite3
from contextlib import contextmanager

@contextmanager
def get_db(db_path='mydb.sqlite'):
    conn = sqlite3.connect(db_path)
    conn.row_factory = sqlite3.Row # Dict-like rows
    try:
        yield conn
    except Exception:
        conn.rollback()
        raise
    finally:

```

```

conn.close()

# Create table and insert data
with get_db() as conn:
    conn.execute('''CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )''')
    conn.executemany(
        'INSERT OR IGNORE INTO users (name, email) VALUES (?, ?)',
        [('Alice', 'alice@example.com'), ('Bob', 'bob@example.com')]
    )

# Query data
with get_db() as conn:
    cursor = conn.execute(
        'SELECT * FROM users WHERE name LIKE ?', ('%Ali%',)
    )
    for row in cursor.fetchall():
        print(dict(row)) # {'id': 1, 'name': 'Alice', ...}

# Using SQLAlchemy ORM
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import DeclarativeBase, Session

class Base(DeclarativeBase): pass
class User(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True)
    name = Column(String)

```

Q81. How do you automate Excel files with openpyxl?

openpyxl reads and writes Excel files with support for formatting, formulas, and charts.

```

from openpyxl import Workbook, load_workbook
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from openpyxl.utils import get_column_letter
from openpyxl.chart import BarChart, Reference

# Create workbook
wb = Workbook()
ws = wb.active
ws.title = 'Sales Report'

# Headers with formatting
headers = ['Region', 'Q1', 'Q2', 'Q3', 'Q4', 'Total']
for col, header in enumerate(headers, 1):

```

```
cell = ws.cell(row=1, column=col, value=header)
cell.font = Font(bold=True, color='FFFFFF')
cell.fill = PatternFill('solid', fgColor='1a1a2e')
cell.alignment = Alignment(horizontal='center')

# Data rows
data = [
    ['North', 100, 120, 130, 150],
    ['South', 90, 110, 105, 130],
    ['East', 80, 95, 120, 140],
]
for row_idx, row in enumerate(data, 2):
    for col_idx, value in enumerate(row, 1):
        ws.cell(row=row_idx, column=col_idx, value=value)
# Sum formula
ws.cell(row=row_idx, column=6,
value=f'=SUM(B{row_idx}:E{row_idx})')

# Auto-adjust column widths
for col in ws.columns:
    max_len = max(len(str(cell.value or '')) for cell in col)
    ws.column_dimensions[col[0].column_letter].width = max_len + 2

wb.save('sales_report.xlsx')
print('Excel file saved!')
```

Section 10: Database & File Format Automation (Q82–Q90)

Q82. How do you use pandas for data automation?

pandas provides powerful tools for loading, transforming, and exporting data from various sources.

```
import pandas as pd
from pathlib import Path

# Read multiple formats
df_csv = pd.read_csv('data.csv')
df_xlsx = pd.read_excel('data.xlsx', sheet_name='Sheet1')
df_json = pd.read_json('data.json')

# Data manipulation
df = pd.DataFrame({
    'name': ['Alice', 'Bob', 'Carol', 'Dave'],
    'sales': [100, 150, 80, 200],
    'region': ['North', 'South', 'North', 'West']
})

# Filter, group, aggregate
north = df[df['region'] == 'North']
by_region = df.groupby('region')['sales'].agg(['sum', 'mean', 'count'])
top2 = df.nlargest(2, 'sales')

print(by_region)

# Apply transformations
df['sales_normalized'] = (df['sales'] - df['sales'].mean()) / df['sales'].std()
df['bonus'] = df['sales'].apply(lambda x: x * 0.1 if x > 100 else 0)

# Export multiple sheets
with pd.ExcelWriter('report.xlsx', engine='openpyxl') as writer:
    df.to_excel(writer, sheet_name='Data', index=False)
    by_region.to_excel(writer, sheet_name='Summary')

# Merge/join DataFrames
regions = pd.DataFrame({'region': ['North', 'South', 'West'],
    'manager': ['Alice', 'Bob', 'Carol']})
merged = df.merge(regions, on='region', how='left')
```

Q83. How do you parse and manipulate XML files?

Use `xml.etree.ElementTree` for simple XML or `lxml` for advanced XPath and namespace support.

```
import xml.etree.ElementTree as ET

xml_string = '''<?xml version="1.0"?>
<catalog>
<book id="bk001">
```

```

<author>Author 1</author>
<title>Python Cookbook</title>
<price>39.95</price>
</book>
<book id="bk002">
<author>Author 2</author>
<title>Clean Code</title>
<price>29.99</price>
</book>
</catalog>'''

tree = ET.ElementTree(ET.fromstring(xml_string))
root = tree.getroot()

# Iterate elements
for book in root.findall('book'):
    book_id = book.get('id')
    title = book.find('title').text
    price = float(book.find('price').text)
    print(f'{book_id}: {title} - ${price}')

# XPath-like queries
for book in root.findall("./book[@id='bk001']"):
    print(book.find('author').text)

# Modify XML
for price_elem in root.iter('price'):
    price_elem.text = str(round(float(price_elem.text) * 1.1, 2))

# Create XML
new_book = ET.SubElement(root, 'book', id='bk003')
ET.SubElement(new_book, 'title').text = 'New Book'
ET.SubElement(new_book, 'price').text = '19.99'
ET.indent(tree, space=' ')
tree.write('output.xml', xml_declaration=True, encoding='utf-8')

```

Q84. How do you work with SQLite for local data storage?

SQLite via sqlite3 is perfect for local scripts needing persistent storage without a separate database server.

```

import sqlite3
from datetime import datetime

# Database abstraction class
class Database:
    def __init__(self, path=':memory:'):
        self.conn = sqlite3.connect(path,
            detect_types=sqlite3.PARSE_DECLTYPES)
        self.conn.row_factory = sqlite3.Row
        self._create_tables()

```



```

def _create_tables(self):
    self.conn.executescript('''
CREATE TABLE IF NOT EXISTS jobs (
id INTEGER PRIMARY KEY AUTOINCREMENT,
name TEXT NOT NULL,
status TEXT DEFAULT 'pending',
result TEXT,
started TIMESTAMP,
finished TIMESTAMP
);
CREATE INDEX IF NOT EXISTS idx_status ON jobs(status);
''')

def add_job(self, name):
    cur = self.conn.execute(
'INSERT INTO jobs (name) VALUES (?)', (name,))
    self.conn.commit()
    return cur.lastrowid

def update_job(self, job_id, status, result=None):
    self.conn.execute(
'UPDATE jobs SET status=?, result=?, finished=? WHERE id=?',
(status, result, datetime.now(), job_id))
    self.conn.commit()

def get_pending(self):
    return self.conn.execute(
"SELECT * FROM jobs WHERE status='pending'").fetchall()

db = Database('jobs.db')
jid = db.add_job('data_export')
db.update_job(jid, 'completed', 'Exported 1000 rows')
for job in db.get_pending():
    print(dict(job))

```

Q85. How do you process large files efficiently?

Use generators, chunked reading, and streaming to process large files without loading them into memory.

```

import csv
from pathlib import Path

# Generator to read CSV in chunks
def read_csv_chunks(filepath, chunk_size=1000):
    with open(filepath, 'r') as f:
        reader = csv.DictReader(f)
        chunk = []
        for row in reader:
            chunk.append(row)

```

```

if len(chunk) >= chunk_size:
    yield chunk
    chunk = []
if chunk:
    yield chunk

def process_chunk(chunk):
    return sum(float(row.get('value', 0)) for row in chunk)

total = 0
for chunk in read_csv_chunks('large_data.csv'):
    total += process_chunk(chunk)
print(f'Processed chunk, running total: {total}')

# Memory-mapped file reading
import mmap
def count_occurrences(filepath, pattern):
    with open(filepath, 'r') as f:
        with mmap.mmap(f.fileno(), 0, access=mmap.ACCESS_READ) as mm:
            count = 0
            pos = 0
            enc_pattern = pattern.encode()
            while True:
                idx = mm.find(enc_pattern, pos)
                if idx == -1:
                    break
                count += 1
                pos = idx + 1
            return count

# Generator pipeline
def read_lines(filename):
    with open(filename) as f:
        yield from f

def filter_lines(lines, pattern):
    for line in lines:
        if pattern in line:
            yield line.strip()

pipeline = filter_lines(read_lines('app.log'), 'ERROR')
errors = list(pipeline)

```

Q86. How do you use Redis for caching in automation scripts?

Redis provides fast in-memory caching and pub/sub messaging for automation workflows.

```

import redis
import json
import time

```

```

import functools

r = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)

# Basic operations
r.set('key', 'value')
r.set('key', 'value', ex=3600) # expires in 1 hour
value = r.get('key')

r.setex('temp_key', 60, 'expires soon') # 60 seconds TTL

# Hash (like dict)
r.hset('user:1001', mapping={'name': 'Alice', 'email': 'a@ex.com'})
user = r.hgetall('user:1001') # {'name': 'Alice', 'email': '...'}

# List (queue)
r.rpush('queue', 'task1', 'task2')
task = r.lpop('queue')

# Cache decorator
def redis_cache(ttl=300):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            key = f'cache:{func.__name__}:{args}:{kwargs}'
            if cached := r.get(key):
                return json.loads(cached)
            result = func(*args, **kwargs)
            r.setex(key, ttl, json.dumps(result))
            return result
        return wrapper
    return decorator

@redis_cache(ttl=600)
def expensive_query(user_id):
    time.sleep(2) # Simulate DB query
    return {'user_id': user_id, 'data': 'result'}

print(expensive_query(42)) # Slow first time
print(expensive_query(42)) # Fast from cache

```

Q87. How do you work with AWS S3 using boto3?

boto3 is the AWS SDK for Python, enabling automation of S3, EC2, and other AWS services.

```

import boto3
from botocore.exceptions import ClientError
import os

# Create S3 client

```

```

s3 = boto3.client('s3',
aws_access_key_id=os.getenv('AWS_ACCESS_KEY_ID'),
aws_secret_access_key=os.getenv('AWS_SECRET_ACCESS_KEY'),
region_name='us-east-1'
)

BUCKET = 'my-automation-bucket'

# Upload file
def upload_file(local_path, s3_key):
try:
s3.upload_file(local_path, BUCKET, s3_key)
print(f'Uploaded {local_path} to s3://{BUCKET}/{s3_key}')
except ClientError as e:
print(f'Upload failed: {e}')

# Download file
def download_file(s3_key, local_path):
try:
s3.download_file(BUCKET, s3_key, local_path)
except ClientError as e:
if e.response['Error']['Code'] == '404':
print('File not found')

# List objects
def list_objects(prefix=''):
paginator = s3.get_paginator('list_objects_v2')
pages = paginator.paginate(Bucket=BUCKET, Prefix=prefix)
for page in pages:
for obj in page.get('Contents', []):
print(f'{obj["Key"]} ({obj["Size"]} bytes)')

# Upload with metadata
s3.put_object(
Bucket=BUCKET,
Key='data/report.json',
Body=b'{"key": "value"}',
ContentType='application/json',
Metadata={'source': 'automation-script', 'version': '1.0'}
)

```

Q88. How do you use Python to interact with Docker?

The docker SDK for Python lets you manage containers, images, and networks programmatically.

```

import docker

client = docker.from_env()

# List running containers

```

```

for container in client.containers.list():
    print(f'{container.name}: {container.status}')

# Run a container
container = client.containers.run(
    'python:3.11',
    command='python -c "print(42)"',
    remove=True, # Auto-remove after exit
    detach=False,
    mem_limit='128m',
    cpu_quota=50000
)
print(container.decode()) # 42

# Run detached container
container = client.containers.run(
    'nginx:latest',
    detach=True,
    ports={'80/tcp': 8080},
    name='my-nginx',
    volumes={'/host/html': {'bind': '/usr/share/nginx/html', 'mode': 'ro'}}
)

# Container operations
container.pause()
container.unpause()
logs = container.logs(tail=100).decode()
stats = container.stats(stream=False)
container.stop(timeout=10)
container.remove()

# Build image from Dockerfile
image, logs = client.images.build(
    path='.',
    tag='myapp:latest',
    rm=True # Remove intermediate containers
)

```

Q89. How do you monitor and alert on file system changes?

Use the watchdog library to monitor directories for file creation, modification, and deletion.

```

from watchdog.observers import Observer
from watchdog.events import (FileSystemEventHandler,
                             FileCreatedEvent, FileModifiedEvent,
                             FileDeletedEvent)
import logging
import time

logging.basicConfig(level=logging.INFO)

```

```

class AutoProcessor(FileSystemEventHandler):
    def __init__(self, process_fn=None):
        self.process_fn = process_fn

    def on_created(self, event):
        if not event.is_directory:
            logging.info(f'New file: {event.src_path}')
            if self.process_fn:
                self.process_fn(event.src_path)

    def on_modified(self, event):
        if not event.is_directory:
            logging.info(f'Modified: {event.src_path}')

    def on_deleted(self, event):
        logging.info(f'Deleted: {event.src_path}')

    def on_moved(self, event):
        logging.info(f'Moved: {event.src_path} -> {event.dest_path}')

    def process_new_file(path):
        logging.info(f'Processing: {path}')
        # Add your processing logic here

handler = AutoProcessor(process_fn=process_new_file)
observer = Observer()
observer.schedule(handler, path='/watch/this/folder', recursive=True)
observer.start()

try:
    while True:
        time.sleep(1)
except KeyboardInterrupt:
    observer.stop()
observer.join()

```

Q90. How do you parse and generate INI, TOML, and other config formats?

Python supports multiple config formats: configparser for INI, tomlib (3.11+) for TOML, json, and yaml.

```

# TOML (Python 3.11+ built-in, or pip install tomli for older)
import tomlib # Python 3.11+

with open('pyproject.toml', 'rb') as f:
    config = tomlib.load(f)

# Write TOML (pip install tomli-w)
import tomli_w
config = {

```

```

'project': {'name': 'myapp', 'version': '1.0.0'},
'dependencies': ['requests>=2.28', 'click>=8.0']
}
with open('pyproject.toml', 'wb') as f:
    toml_w.dump(config, f)

# INI with configparser
import configparser
config = configparser.ConfigParser()
config['DEFAULT'] = {'timeout': '30', 'retries': '3'}
config['production'] = {'database': 'prod-db', 'debug': 'false'}
config['development'] = {'database': 'dev-db', 'debug': 'true'}
with open('settings.ini', 'w') as f:
    config.write(f)

# Unified config loader
import json, yaml
from pathlib import Path

def load_config(path):
    path = Path(path)
    suffix = path.suffix.lower()
    if suffix == '.json':
        return json.loads(path.read_text())
    elif suffix in ('.yaml', '.yml'):
        import yaml
        return yaml.safe_load(path.read_text())
    elif suffix == '.toml':
        return tomlib.loads(path.read_text())
    elif suffix in ('.ini', '.cfg'):
        config = configparser.ConfigParser()
        config.read(path)
        return {s: dict(config[s]) for s in config.sections()}
    raise ValueError(f'Unsupported format: {suffix}')

```

Section 11: Advanced Scripting Patterns (Q91–Q100)

Q91. How do you write idempotent scripts?

Idempotent scripts can be run multiple times safely with the same result. Always check before acting.

```
import os
import sqlite3
from pathlib import Path

# Idempotent file creation
def ensure_directory(path):
    Path(path).mkdir(parents=True, exist_ok=True) # exist_ok makes it idempotent

# Idempotent database migration
def apply_migration(conn, migration_id, sql):
    conn.execute('''CREATE TABLE IF NOT EXISTS migrations (
    id TEXT PRIMARY KEY, applied_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )''')
    # Check if already applied
    existing = conn.execute(
        'SELECT id FROM migrations WHERE id=?', (migration_id,)
    ).fetchone()
    if existing:
        print(f'Migration {migration_id} already applied, skipping')
        return
    conn.executescript(sql)
    conn.execute('INSERT INTO migrations (id) VALUES (?)', (migration_id,))
    conn.commit()
    print(f'Applied migration: {migration_id}')

conn = sqlite3.connect('app.db')
apply_migration(conn, '001_create_users',
    'CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, name TEXT);')
apply_migration(conn, '001_create_users', # Running again is safe!
    'CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, name TEXT);')

# Idempotent HTTP PUT (vs POST)
import requests
# PUT is idempotent: calling it multiple times has same effect
requests.put('https://api.example.com/users/123', json={'name': 'Alice'})
requests.put('https://api.example.com/users/123', json={'name': 'Alice'}) # Safe
```

Q92. How do you write a plugin system in Python?

Use `importlib` or `entry_points` to dynamically discover and load plugins at runtime.

```
import importlib
import importlib.util
from pathlib import Path
from abc import ABC, abstractmethod
```



```

# Base plugin interface
class BasePlugin(ABC):
    name: str
    version: str = '1.0'

    @abstractmethod
    def execute(self, data):
        pass

# Plugin manager
class PluginManager:
    def __init__(self, plugin_dir='plugins'):
        self.plugins = {}
        self.plugin_dir = Path(plugin_dir)

    def discover(self):
        for py_file in self.plugin_dir.glob('*.py'):
            if py_file.stem.startswith('_'):
                continue
            spec = importlib.util.spec_from_file_location(
                py_file.stem, py_file)
            module = importlib.util.module_from_spec(spec)
            spec.loader.exec_module(module)
            for name in dir(module):
                obj = getattr(module, name)
                if (isinstance(obj, type) and
                    issubclass(obj, BasePlugin) and
                    obj is not BasePlugin):
                    instance = obj()
                    self.plugins[instance.name] = instance
            print(f'Loaded plugin: {instance.name}')

    def run(self, plugin_name, data):
        if plugin_name not in self.plugins:
            raise ValueError(f'Plugin not found: {plugin_name}')
        return self.plugins[plugin_name].execute(data)

pm = PluginManager()
pm.discover() # Auto-discover all plugins in ./plugins/

```

Q93. How do you implement a pipeline pattern for data processing?

Chain functions or generators together in a pipeline for clean, composable data transformations.

```

from typing import Callable, Iterable, TypeVar
import functools

T = TypeVar('T')

```

```

# Generator pipeline
def pipeline(*generators):
    """Chain generators: data flows through each stage."""
    def _pipeline(data):
        result = data
        for gen in generators:
            result = gen(result)
        return result
    return _pipeline

# Pipeline stages (all are generators)
def read_lines(filename):
    with open(filename) as f:
        yield from f

def strip_lines(lines):
    for line in lines:
        yield line.strip()

def filter_empty(lines):
    for line in lines:
        if line:
            yield line

def filter_comments(lines):
    for line in lines:
        if not line.startswith('#'):
            yield line

def to_upper(lines):
    for line in lines:
        yield line.upper()

# Compose pipeline
process = pipeline(strip_lines, filter_empty, filter_comments, to_upper)

# Functional pipeline with reduce
def compose(*funcs):
    """Compose single-value transforms."""
    return functools.reduce(lambda f, g: lambda x: g(f(x)), funcs)

double = lambda x: x * 2
add_one = lambda x: x + 1
to_str = lambda x: str(x)

transform = compose(double, add_one, to_str)
print(transform(5)) # '11'

```

Q94. How do you use decorators for cross-cutting concerns in scripts?

Decorators add logging, timing, caching, rate-limiting, and auth checks without cluttering business logic.

```
import functools
import time
import logging

# Timing decorator
def timed(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        logging.info(f'{func.__name__} took {elapsed:.3f}s')
        return result
    return wrapper

# Rate limiting decorator
def rate_limit(calls_per_second):
    min_interval = 1.0 / calls_per_second
    def decorator(func):
        last_call = [0.0]
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            elapsed = time.time() - last_call[0]
            if elapsed < min_interval:
                time.sleep(min_interval - elapsed)
            last_call[0] = time.time()
            return func(*args, **kwargs)
        return wrapper
    return decorator

# Memoize decorator
def memoize(func):
    cache = {}
    @functools.wraps(func)
    def wrapper(*args):
        if args not in cache:
            cache[args] = func(*args)
        return cache[args]
    return wrapper

@timed
@rate_limit(2) # Max 2 calls per second
@memoize
def fetch_user(user_id):
    time.sleep(0.1) # Simulate API call
    return {'id': user_id, 'name': f'User_{user_id}'}

for i in [1, 2, 1, 3, 2]:
    print(fetch_user(i))
```

Q95. How do you write a configuration management script using Python?

Build idempotent configuration management: install packages, write files, enable services in a declarative way.

```
import subprocess
import shutil
import os
from pathlib import Path

class Configurator:
    def __init__(self, dry_run=False):
        self.dry_run = dry_run
        self.changes = []

    def _run(self, cmd, desc):
        print(f' {desc}')
        if not self.dry_run:
            subprocess.run(cmd, check=True, capture_output=True)
        self.changes.append(desc)

    def ensure_package(self, package):
        if shutil.which(package):
            print(f' [OK] {package} already installed')
            return
        self._run(['apt-get', 'install', '-y', package],
                  f'Install {package}')

    def ensure_file(self, path, content, mode=0o644):
        p = Path(path)
        if p.exists() and p.read_text() == content:
            print(f' [OK] {path} up to date')
            return
        print(f' Writing {path}')
        if not self.dry_run:
            p.parent.mkdir(parents=True, exist_ok=True)
            p.write_text(content)
            p.chmod(mode)
        self.changes.append(f'Write {path}')

    def ensure_user(self, username):
        result = subprocess.run(['id', username], capture_output=True)
        if result.returncode == 0:
            print(f' [OK] User {username} exists')
            return
        self._run(['useradd', '-m', '-s', '/bin/bash', username],
                  f'Create user {username}')

config = Configurator(dry_run=True) # Test without changes
config.ensure_file('/etc/myapp/config.yaml',
                  'host: localhost\nport: 8080\n')
print(f'Changes: {config.changes}')
```

Q96. How do you implement health checks for automation scripts?

Implement health check functions that verify dependencies, connections, and resources before running.

```
import socket
import subprocess
import shutil
import os
from dataclasses import dataclass
from typing import Callable

@dataclass
class HealthCheck:
    name: str
    fn: Callable
    critical: bool = True

    def check_command(cmd):
    return lambda: shutil.which(cmd) is not None

    def check_port(host, port, timeout=3):
    def _check():
    try:
    with socket.create_connection((host, port), timeout):
    return True
    except (socket.timeout, ConnectionRefusedError):
    return False
    return _check

    def check_disk_space(path='/', min_gb=1):
    def _check():
    stat = os.statvfs(path)
    free_gb = (stat.f_bavail * stat.f_frsize) / (1024**3)
    return free_gb >= min_gb
    return _check

    def check_env_var(name):
    return lambda: bool(os.environ.get(name))

    checks = [
    HealthCheck('git', check_command('git')),
    HealthCheck('database', check_port('localhost', 5432)),
    HealthCheck('disk space', check_disk_space(min_gb=2)),
    HealthCheck('API_KEY', check_env_var('API_KEY')),
    ]

    def run_health_checks(checks):
    all_pass = True
    for check in checks:
    passed = check.fn()
    status = 'PASS' if passed else 'FAIL'
```

```

print(f'[{status}] {check.name}')
if not passed and check.critical:
    all_pass = False
return all_pass

if not run_health_checks(checks):
    raise SystemExit('Health checks failed')

```

Q97. How do you package and distribute a Python script?

Use pyproject.toml with setuptools or Hatch to package scripts with proper entry points and dependencies.

```

# pyproject.toml
# [build-system]
# requires = ["setuptools>=68", "wheel"]
# build-backend = "setuptools.backends.legacy:build"
#
# [project]
# name = "my-automation-tool"
# version = "1.0.0"
# description = "Automation toolkit"
# requires-python = ">=3.11"
# dependencies = [
#     "requests>=2.28",
#     "click>=8.0",
#     "python-dotenv>=1.0",
# ]
#
# [project.scripts]
# mytool = "mypackage.cli:main"

# cli.py - entry point
import click
from pathlib import Path

@click.group()
@click.version_option('1.0.0')
def main():
    """My Automation Tool"""
    pass

@main.command()
@click.argument('files', nargs=-1, type=click.Path(exists=True))
@click.option('-o', '--output', default='output.txt')
def process(files, output):
    """Process input files."""
    for file in files:
        click.echo(f'Processing: {file}')
        click.echo(f'Output: {output}')

```

```

if __name__ == '__main__':
    main()

# Build and install:
# pip install build
# python -m build # creates dist/
# pip install dist/*.whl
# mytool --help

```

Q98. How do you write unit tests for automation scripts?

Use pytest with mocks to test automation scripts without actually calling external services.

```

import pytest
from unittest.mock import patch, MagicMock, call
import subprocess

# Code under test
def run_backup(source, destination):
    result = subprocess.run(
        ['rsync', '-av', source, destination],
        capture_output=True, text=True, check=True
    )
    return result.stdout

def fetch_config(url):
    import requests
    resp = requests.get(url, timeout=10)
    resp.raise_for_status()
    return resp.json()

# Tests
class TestBackup:
    @patch('subprocess.run')
    def test_run_backup_success(self, mock_run):
        mock_run.return_value = MagicMock(
            stdout='transferred 5 files', returncode=0)
        result = run_backup('/src/', '/dst/')
        mock_run.assert_called_once_with(
            ['rsync', '-av', '/src/', '/dst/'],
            capture_output=True, text=True, check=True)
        assert 'transferred' in result

    @patch('subprocess.run')
    def test_run_backup_failure(self, mock_run):
        mock_run.side_effect = subprocess.CalledProcessError(1, 'rsync')
        with pytest.raises(subprocess.CalledProcessError):
            run_backup('/src/', '/dst/')

    @patch('requests.get')

```

```
def test_fetch_config(self, mock_get):
    mock_get.return_value = MagicMock(json=lambda: {'key': 'value'})
    result = fetch_config('https://example.com/config')
    assert result == {'key': 'value'}
```

Q99. How do you profile and optimize a Python script?

Use cProfile, line_profiler, and memory_profiler to identify bottlenecks in your scripts.

```
import cProfile
import pstats
import io
import time

# cProfile - function-level profiling
def slow_function():
    time.sleep(0.1)
    return sum(i**2 for i in range(100000))

# Profile with cProfile
profiler = cProfile.Profile()
profiler.enable()
slow_function()
profiler.disable()

stream = io.StringIO()
stats = pstats.Stats(profiler, stream=stream)
stats.sort_stats('cumulative')
stats.print_stats(10) # Top 10
print(stream.getvalue())

# Or use command line: python -m cProfile -s cumtime script.py

# timeit for micro-benchmarks
import timeit
time1 = timeit.timeit('".join(str(n) for n in range(100))', number=10000)
time2 = timeit.timeit('".join(map(str, range(100)))', number=10000)
print(f'Generator: {time1:.4f}s, map: {time2:.4f}s')

# memory_profiler decorator usage (pip install memory-profiler)
# @memory_profiler.profile
# def memory_intensive():
#     return [i for i in range(1000000)]

# Simple manual benchmark
def benchmark(func, iterations=1000):
    start = time.perf_counter()
    for _ in range(iterations):
        func()
    return (time.perf_counter() - start) / iterations
```


Q100. How do you build a complete automation script with best practices?

Combine all patterns: logging, config, error handling, retry, CLI, and health checks into a robust script.

[illegible]

